Universidad San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Curso: Sistemas Operativos 2
Sección: A
Ingeniero: Otto Alejandro Soto Macal

MANUAL TÉCNICO

Proyecto Final - RedXela

Estudiante: Herberth Julian Reyes Pacajoj
Registro Académico: 202230236

Quetzaltenango, octubre 2026

Contenido

1. Introducción 4
    1.1 Propósito de este manual 4
    1.2 El problema 4
    1.3 Qué se construyó 4
    1.4 Conceptos aplicados 5
    1.5 El proyecto en cifras 5
2. Arquitectura general 6
    2.1 Visión de conjunto 6
    2.2 Stack tecnológico y justificación 7
    2.3 Estructura de paquetes 7
    2.4 Comunicación con la interfaz 8
    2.5 Raíz de composición y reinicio de la simulación 8
    2.6 Mapa de hilos y mecanismos de sincronización 9
3. Modelo del dominio 11
    3.1 Recursos compartidos 11
    3.2 Niveles de servicio 11
    3.3 Tipos de mercancía y recetas 11
    3.4 Catálogo de productos 12
    3.5 Pedidos y clientes 13
    3.6 Tiempos de simulación 13
4. Reloj simulado 14
5. Planificación 15
    5.1 El problema de la planificación 15
    5.2 La lista de espera y el despachador 15
    5.3 Las cinco políticas 15
    5.4 Pedidos que esperan stock 16
    5.5 Generador automático de pedidos 16
    5.6 Estadísticas 17
    5.7 Comparador de políticas 17
6. Sincronización y exclusión mutua 20
    6.1 Principios 20
    6.2 ResourcePool: un monitor que hace de semáforo 20
    6.3 WaitingList 22
    6.4 Lectores y escritores: Inventory y Warehouse 22
    6.5 Resumen de mecanismos 23
    6.6 Cómo se prueba la concurrencia 23
7. Flujo de un pedido 24
    7.1 Admisión 24
    7.2 Etapas y modelo de retención 25
    7.3 Estado observable 25
8. Comunicación entre áreas: Recepción e Inventario 26

8.1 Recepción (productores)..... 26
8.2 El buffer acotado..... 27
8.3 Inventario (consumidor)..... 27
9. Administración de almacenamiento..... 28
9.1 Dimensiones y reglas..... 28
9.2 Estrategias de colocación y salida..... 29
9.3 Almacén lleno..... 29
9.4 Consistencia con el inventario..... 29
9.5 Visualización..... 29
10. Deadlock..... 30
10.1 El requisito..... 30
10.2 El escenario..... 30
10.3 Las cuatro condiciones de Coffman..... 31
10.4 Detección..... 31
10.5 Confirmación por estabilidad..... 33
10.6 Prevención..... 33
10.7 Recuperación manual por apropiación..... 33
11. Interfaz web..... 34
11.1 Qué muestra..... 34
11.2 Atomic Design..... 34
11.3 Flujo de datos..... 35
11.4 Accesibilidad y presentación..... 35
12. API REST y WebSocket..... 36
13. Estrategia de pruebas y resultados..... 37
13.1 Cobertura..... 37
13.2 Técnicas para probar concurrencia..... 37
13.3 Pruebas de mutación manuales..... 37
13.4 Prueba de aguante..... 38
13.5 Defectos que encontraron las pruebas..... 38
14. Decisiones de diseño y alternativas descartadas..... 40
15. Limitaciones y trabajo futuro..... 41
Apéndice A. Compilación, ejecución y pruebas..... 42
Apéndice B. Glosario..... 43

1. Introducción

1.1 Propósito de este manual

Este manual describe la **infraestructura** y las **decisiones de diseño** de REDXela, el simulador del centro de distribución desarrollado como proyecto final del curso Sistemas Operativos 2. Aquí explica *cómo está construido* el sistema y, sobre todo, *por qué* se construyó así, relacionando cada decisión con los conceptos teóricos del curso: planificación, exclusión mutua, semáforos y monitores, prevención y detección de deadlock, comunicación entre procesos y administración de almacenamiento.

Cada sección sigue el mismo esquema: **qué problema se resuelve → qué concepto teórico aplica → cómo se implementó (con la clase responsable) → por qué se descartaron las alternativas**.

1.2 El problema

REDXela es un centro de distribución de comercio electrónico que recibe, procesa y despacha un volumen alto y variable de pedidos. Todos compiten por un conjunto limitado de recursos compartidos, y una mala gestión puede producir retrasos, pedidos perdidos o, peor, que el sistema completo quede detenido esperando recursos que nunca se liberan. El centro cuenta con:

- 6 estaciones de empaque, 2 áreas de carga (para camiones), 10 encargados de bodega, 3 montacargas y 4 estaciones de control de calidad;
- 1 sistema de escaneo de códigos, **compartido** entre Recepción y Despacho (el recurso clave del deadlock);
- un almacén cuya capacidad define el estudiante.

Cada pedido pertenece a un cliente con un nivel de servicio (Exprés, Prioritario, Estándar, Programado o Económico) que determina cuánto puede esperar, y contiene productos de distintos tipos de mercancía, que determinan qué recursos necesita.

1.3 Qué se construyó

- **Simulación concurrente en Java** con hilos reales: un hilo por pedido en proceso, un hilo por camión, y hilos propios para el despachador, el generador, el consumidor de Inventario, el detector de deadlock y el comparador de políticas.
- **Cinco políticas de atención** intercambiables en caliente (FIFO, prioridad estricta, prioridad con envejecimiento, EDF y SJF) y un **comparador** que las enfrenta con la misma carga.
- **Recepción e Inventario** como áreas independientes que solo se comunican por mensajes, mediante un buffer acotado (productor/consumidor).
- **Almacén con asignación no contigua** (análogo a la paginación de memoria), con 240 ubicaciones y mapa visual.
- **Deadlock deliberado**: se provoca, se detecta con el algoritmo de recursos de varias instancias, se explica en pantalla (recursos, operaciones y las cuatro condiciones de Coffman) y se resuelve manualmente por apropiación; además se puede prevenir con un orden global de recursos.

- **Interfaz web en tiempo real** (React + TypeScript) que muestra recursos, pedidos en proceso, lista de espera con su criterio de orden, estadísticas, inventario, almacén y deadlock, y permite registrar pedidos y mercadería a mano.
- **Generador automático** de pedidos de todos los niveles, con llegadas de Poisson y semilla reproducible.

1.4 Conceptos aplicados

La siguiente tabla indica dónde se aplica cada concepto y en qué sección de este manual se explica.

Tabla 1. Dónde se aplica cada concepto del curso.

Concepto	Clase(s) principal(es)	Sección
Planificación	`SchedulingPolicy`, `WaitingList`, `Dispatcher`, `BenchmarkRunner`	5
Exclusión mutua y monitores	`ResourcePool` (ReentrantLock + Condition por espera)	6.2
Semáforos	`ResourcePool` = semáforo de conteo con identidad y prioridad	6.2
Lectores / escritores	`Inventory`, `Warehouse` (ReentrantReadWriteLock)	6.4
Productor / consumidor	`ReceptionBuffer`, `ReceptionService`, `InventoryWorker`	8
Comunicación entre procesos	`ReceptionMessage` por buffer acotado; `stockChanged()`	8
Administración de almacenamiento	`Warehouse`, `PlacementStrategy` (paginación no contigua)	9
Detección de deadlock	`DeadlockAnalyzer`, `DeadlockDetector`	10.4
Prevención de deadlock	`AcquisitionPolicy`, `PreventionMode`, `OperationOrdering`	10.6
Recuperación de deadlock	`ResourcePool.preempt`, `OperationPreemptedException`	10.7

1.5 El proyecto en cifras

Tabla 2. Tamaño y verificación del proyecto.

Aspecto	Valor
Backend (Java 21)	131 archivos Java en `src/main`, ≈7.000 líneas (incluye Javadoc y comentarios)
Frontend (TypeScript)	≈4.200 líneas sin contar tests; 6 átomos, 24 moléculas, 16 organismos, 1 plantilla, 1 página
Pruebas del backend	**352** tests (JUnit 5 + AssertJ), incluidas pruebas de concurrencia y de API con servidor real
Pruebas del frontend	**333** tests (Vitest + Testing Library); `tsc` y lint sin advertencias
Prueba de aguante	12 min a ×600 (≈2 h simuladas), 17.284 pedidos en espera, 256 MB de heap: sin fugas (sección 13.4)

2. Arquitectura general

2.1 Visión de conjunto

REDXela tiene tres capas, y la regla que las gobierna es que **las dependencias solo apuntan hacia el núcleo**: el navegador conoce la API, la API conoce el núcleo, y el núcleo no conoce a nadie por encima. Figura 1 resume la estructura.

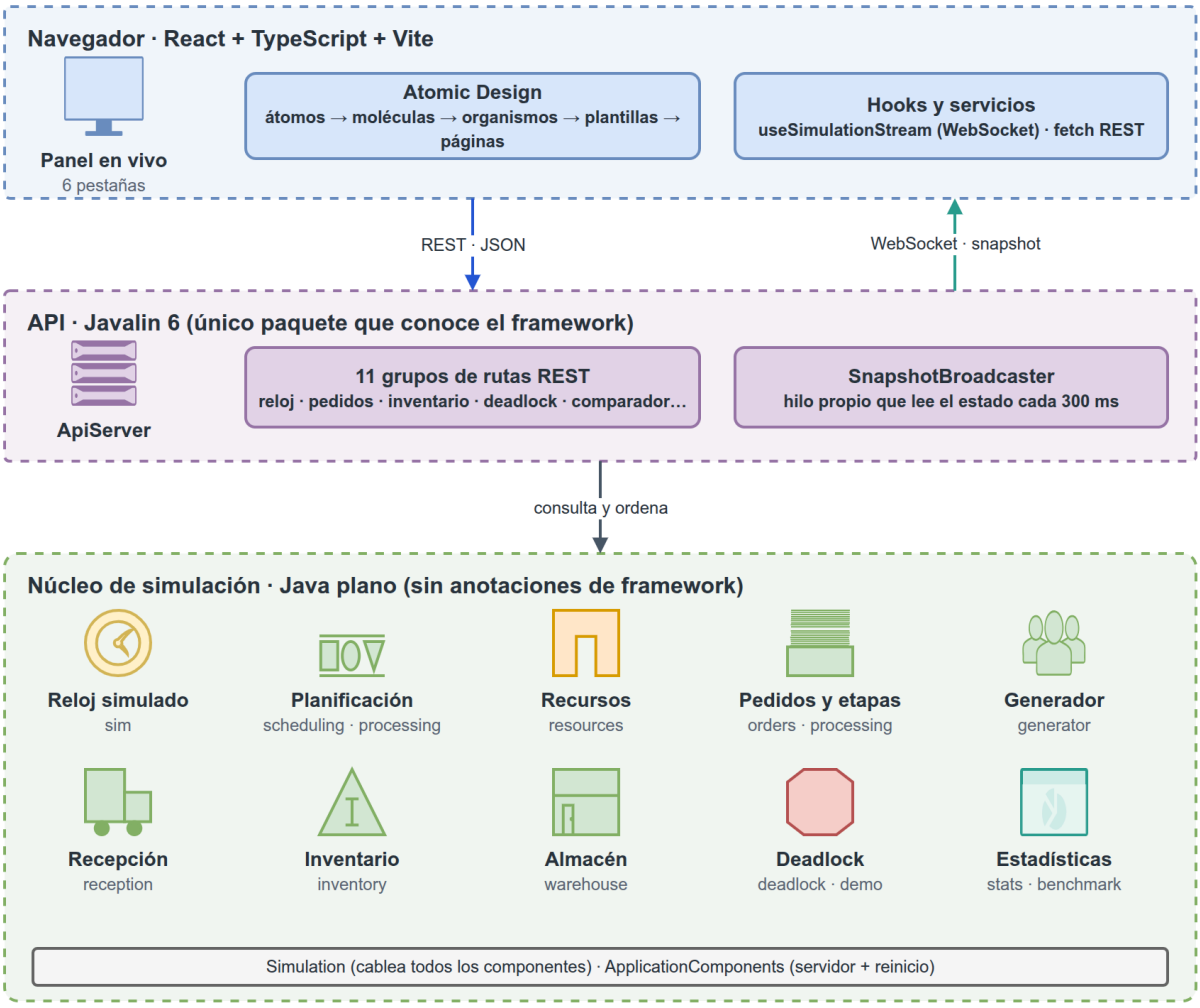


Figura 1. Arquitectura de tres capas: navegador, API y núcleo de simulación.

El **núcleo** es Java plano, sin anotaciones de ningún framework: representa el centro de distribución y no sabe que existe una red. Solo el paquete `api` conoce Javalin. Esta separación tiene consecuencias prácticas: los tests del núcleo no levantan ningún servidor ni contexto, el comparador de políticas corre simulaciones completas sin HTTP, y explicar la concurrencia es más sencillo porque **los únicos hilos son los que el proyecto crea a propósito**.

2.2 Stack tecnológico y justificación

Tabla 3. Tecnologías elegidas y motivo.

Capa	Decisión	Motivo
Lenguaje	Java 21	Lo exige el enunciado («hilos y mecanismos reales de sincronización»). `java.util.concurrent.locks` ofrece los bloques para construir monitores propios.
Build	Maven; JAR único con Shade	Un solo artefacto ejecutable (`java -jar`). `mvn package` construye también el frontend.
API	Javalin 6.7 + Jackson	API mínima (REST + 1 WebSocket) sobre Jetty. Se descartó Spring: su contenedor, proxies y pools de hilos propios oscurecen justo lo que se debe demostrar (qué hilos existen y quién los crea).
Tiempo real	WebSocket con *snapshot* completo cada 300 ms	Más simple y robusto que enviar diferencias: no depende del orden de los mensajes y un cliente que se reconecta se pone al día con el siguiente.
Frontend	React + TypeScript + Vite	Muchos paneles en vivo; el tipado evita errores en el contrato JSON con el backend.
Arquitectura UI	Atomic Design + CSS Modules	Componentes pequeños, reutilizables y con reglas de dependencia verificables (sección 11).
Pruebas	JUnit 5, AssertJ, Vitest, Testing Library	Pruebas deterministas de concurrencia (latches y esperas con tiempo máximo).
Persistencia	Ninguna (todo en memoria)	No es el foco del curso. Las cuentas de cliente sobreviven a un reinicio de la simulación, no al de la JVM.

> **Decisión: Javalin y no Spring**
> Spring crea y administra hilos por su cuenta (pool de Tomcat/Jetty, ejecutores, proxies). Con Javalin la aplicación se reduce a un servidor Jetty y a los hilos que el propio código crea con nombre (`dispatcher`, `order-N`, `truck-N`, `inventory-worker`, `deadlock-detector`…).

2.3 Estructura de paquetes

Tabla 4. Paquetes del backend (`com.redxela`) y su responsabilidad.

Paquete	Responsabilidad
`domain`	Vocabulario del dominio: recursos (`ResourceType`), niveles (`ServiceLevel`), tipos de mercancía, pedidos y estados. Inmutable.
`sim`	`SimClock`: reloj simulado con aceleración y pausa.
`resources`	`ResourcePool` (monitor), `ResourceRegistry`, `Operation`, expulsión por apropiación.
`scheduling`	`SchedulingPolicy` (las 5 políticas) y `WaitingList`.
`processing`	Despachador, planificador, procesador de pedidos, etapas y traspaso de recursos (`OrderWorkflow`).
`orders`, `catalog`, `customers`	Alta de pedidos con validación, catálogo de 15 productos, cuentas de cliente.
`generator`	Generador automático: *qué* llega (`WorkloadGenerator`) y *cuándo* (`OrderGenerator`).

Paquete	Responsabilidad
`reception`	Camiones de Recepción: hilos que descargan y reportan.
`inventory`	`Inventory`, buffer de recepción acotado y `InventoryWorker` (consumidor).
`warehouse`	Almacén con ubicaciones, lotes y estrategias de colocación.
`deadlock`	Analizador, detector, prevención y resolución.
`demo`	`DeadlockScenario`: coreografía que provoca el deadlock a propósito.
`stats`, `benchmark`	Estadísticas en vivo y comparador de políticas.
`common`	Excepciones compartidas (`StateConflictException`).
`api`	Único paquete que conoce Javalin: rutas, WebSocket y *snapshot*.
raíz	`Simulation` (cableado), `ApplicationComponents` (servidor + reinicio) y `RedxelaApplication` (punto de entrada).

2.4 Comunicación con la interfaz

La interfaz usa dos canales. Las **acciones** del usuario (registrar un pedido, cambiar la política, resolver un deadlock…) viajan por REST y reciben respuestas con códigos explícitos: 400 si el argumento es inválido, 409 si la acción ya no corresponde al estado del sistema (por ejemplo, resolver un deadlock que se deshizo solo), 202 si la acción es asíncrona. El **estado** viaja por un único WebSocket (`/ws`): el `SnapshotBroadcaster`, con su propio hilo, arma cada 300 ms una foto completa del sistema y la envía a todos los clientes.

> **Decisión: snapshot completo en lugar de diferencias**
> Enviar solo los cambios exigiría un protocolo de eventos ordenado y una forma de resincronizar al cliente que pierde un mensaje. Con la foto completa el cliente siempre queda en un estado coherente con solo recibir el siguiente mensaje, y reconectarse tras un reinicio del servidor no requiere ningún trato especial. El costo —≈18 KB por mensaje— se mantiene acotado porque el snapshot incluye solo los primeros 50 pedidos de la lista de espera más el total (medido: 18 KB con 17.000 pedidos en espera).

2.5 Raíz de composición y reinicio de la simulación

`Simulation` crea todos los componentes y los conecta entre sí: reloj, recursos, planificador, procesador, inventario, almacén, detector, generador, comparador y rutas. Es el único lugar que conoce a todos; el resto de las clases solo reciben por su constructor lo que necesitan. `ApplicationComponents` une una `Simulation` con el servidor HTTP y sabe reiniciarla. Los tests de integración usan esta misma clase, de modo que prueban el cableado real.

Reiniciar la simulación (botón «Reiniciar simulación») significa cerrar el servidor y la simulación actuales y levantar otros nuevos en el mismo puerto. Se conservan las cuentas de cliente (son datos maestros, como el catálogo); todo lo demás vuelve al estado inicial, incluidos el reloj (×60) y la política.

Decisión: reconstruir en lugar de limpiar

Limpiar uno por uno decenas de estados con hilos en vuelo —pedidos a mitad de una etapa, camiones esperando recursos, un deadlock vigente— habría sido una fuente de carreras: cada limpieza tendría que sincronizarse con hilos que siguen corriendo. Descartar el objeto completo y construir otro es correcto por construcción. Hay dos detalles: la respuesta `202` sale antes de apagar el servidor (con un margen de 300 ms) para que el navegador no vea una conexión cortada, y un test verifica que los reinicios repetidos no dejan hilos vivos.

2.6 Mapa de hilos y mecanismos de sincronización

Figura 2 muestra qué hilos existen (en morado) y qué objetos compartidos los coordinan (en naranja). Cada flecha es una interacción que, en el código, ocurre bajo un lock concreto; la sección 6 detalla cada mecanismo.

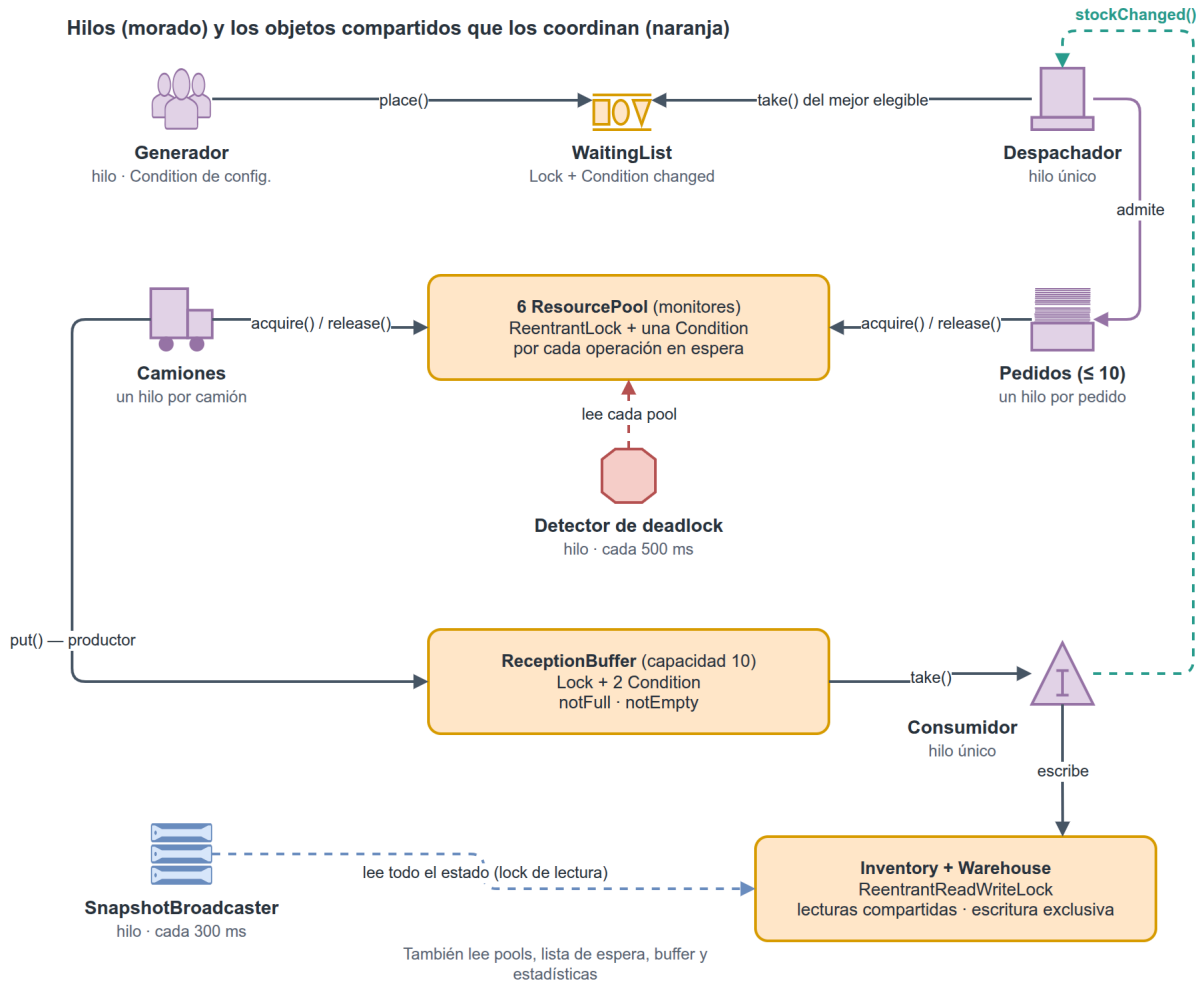


Figura 2. Hilos de la aplicación y los objetos compartidos que los coordinan.

Tabla 5. Hilos de la aplicación.

Hilo	Cantidad	Mecanismo clave
Despachador	1	`WaitingList` (Lock + Condition) y `ResourcePool.awaitAvailableUnit`
Operación de pedido	≤ 10 a la vez (uno por encargado)	`ResourcePool` (ReentrantLock + una Condition por espera)
Camión de recepción	uno por camión (tope de 100 pendientes)	Pools de recursos + productor del buffer
Consumidor de Inventario	1	Buffer acotado (Lock + 2 Condition) + ReadWriteLock
Generador de pedidos	1	`SimClock` + Condition de configuración
Detector de deadlock	1	Lecturas por pool + confirmación por estabilidad
Difusor de snapshots	1	Lecturas con locks breves
Comparador de políticas	1 mientras compara (+ los de la corrida en curso)	Registro inmutable `volatile`; interrupción para cancelar

3. Modelo del dominio

3.1 Recursos compartidos

Las cantidades las fija el enunciado. Cada recurso pertenece a una **etapa** del flujo, y el orden de declaración en `ResourceType` define el **orden global de adquisición** (*rank*): si todas las operaciones piden sus recursos en ese orden, nunca puede formarse una espera circular. Esa propiedad es la base de la prevención de deadlock (sección 10.6).

Tabla 6. Recursos, cantidades, etapa y orden global.

Rank	Recurso	Unidades	Etapa
1	Encargado de bodega	10	Picking (se retiene durante todo el pedido)
2	Montacargas	3	Picking (solo si la mercancía lo requiere)
3	Estación de control de calidad	4	Inspección
4	Estación de empaque	6	Empaque
5	Escáner de códigos	1	Despacho (compartido con Recepción)
6	Área de carga	2	Despacho (compartida con Recepción)

3.2 Niveles de servicio

El tiempo objetivo de cada nivel funciona como un **deadline** sobre la espera: `deadline = llegada + objetivo del nivel`. «Procesamiento casi instantáneo» para Exprés se modela como 1 minuto simulado. Un pedido está *a tiempo* si empieza a atenderse (se le asigna un encargado) dentro de su objetivo.

Tabla 7. Niveles de servicio y mezcla del generador automático.

Nivel	Tipo de envío	Objetivo de espera	Mezcla del generador
1	Exprés	casi instantáneo (1 min)	5 %
2	Prioritario	10 min	15 %
3	Estándar	30 min	35 %
4	Programado	60 min	25 %
5	Económico	120 min	20 %

3.3 Tipos de mercancía y recetas

El nivel de servicio decide *cuándo* se atiende un pedido; el tipo de mercancía decide *qué recursos* necesita. Cada tipo declara su **receta**: la lista ordenada de recursos que usa (sin contar al encargado, que todo pedido necesita). El enunciado pide cinco tipos con combinaciones distintas y al menos un escenario de contención entre dos pedidos por los mismos dos recursos; ambos requisitos se verifican con tests.

Tabla 8. Tipos de mercancía y su receta (en orden global de adquisición).

Tipo	Características	Receta
Estándar	Ropa, libros y accesorios sin requisitos especiales	Empaque → Escáner → Carga
Frágil	Cristalería y cerámica: requiere inspección antes de empacar	Calidad → Empaque → Escáner → Carga
Pesado	Electrodomésticos y muebles: requiere montacargas	Montacargas → Empaque → Escáner → Carga
Perecedero	Alimentos y medicinas: llegan empacados, van directo a despacho	Calidad → Escáner → Carga
Voluminoso	Colchones y bicicletas: montacargas e inspección	Montacargas → Calidad → Empaque → Escáner → Carga

Contención deliberada: *Pesado* y *Voluminoso* se disputan los mismos dos recursos compartidos, el montacargas (3 unidades) y la estación de empaque (6 unidades). Con carga alta, pedidos de ambos tipos llegan a esos pools en momentos cercanos y el sistema debe resolverlo con su política de prioridad. Un pedido con varias líneas usa la **unión** de las recetas de sus líneas, ordenada con el orden global; como todas las recetas son subsecuencias de ese orden, la unión queda bien definida.

3.4 Catálogo de productos

El catálogo es fijo (el enunciado indica que no hace falta crear productos): 15 productos, tres por tipo de mercancía. El *volumen por unidad* (uv) determina cuántas unidades caben en una ubicación del almacén (sección 9).

Tabla 9. Catálogo de productos.

Id	Producto	Tipo	uv
P-001	Camiseta de algodón	Estándar	2
P-002	Novela en tapa blanda	Estándar	2
P-003	Mochila urbana	Estándar	8
P-004	Juego de copas de cristal	Frágil	6
P-005	Jarrón de cerámica	Frágil	10
P-006	Espejo decorativo	Frágil	12
P-007	Refrigeradora 300 L	Pesado	60
P-008	Lavadora de carga frontal	Pesado	50
P-009	Escritorio de madera	Pesado	40
P-010	Caja de medicamentos	Perecedero	1
P-011	Fórmula láctea infantil	Perecedero	3
P-012	Chocolates artesanales	Perecedero	2
P-013	Colchón matrimonial	Voluminoso	80
P-014	Bicicleta de montaña	Voluminoso	45
P-015	Silla de oficina ergonómica	Voluminoso	30

3.5 Pedidos y clientes

Un **pedido** contiene: identificador único (`ORD-0001`), cliente, nivel de servicio (el de su cuenta), líneas (producto, tipo de mercancía y cantidad), total de artículos y estado. Se añadieron la hora y el número de llegada (que desempatan cualquier política) y el contador de reintentos por deadlock. Un **cliente** es una cuenta con nombre y nivel de servicio; la interfaz permite crear cuentas nuevas y existen cinco de ejemplo, una por nivel.

Tabla 10. Estados de un pedido.

Estado	Significado
`WAITING`	En la lista de espera: no hay encargado libre o falta stock. «Esperando stock» **no es un estado guardado**: se deduce del inventario al armar el snapshot (qué productos y cuántas unidades faltan), así nunca queda desactualizado.
`PROCESSING`	Un encargado lo procesa y ya retiene al menos un recurso. Su progreso y tiempo restante se calculan con el reloj simulado.
`COMPLETED`	Despachado: liberó todos sus recursos y sus unidades salieron del inventario.

3.6 Tiempos de simulación

La cantidad de unidades de un pedido es la estimación base del trabajo: **trabajo por etapa = unidades × tiempo por unidad de la etapa**. Todos los valores son configurables (`ProcessingTimes`, `ReceptionTimes`, `InventoryTimes`) y están en tiempo simulado.

Tabla 11. Tiempos por defecto (simulados).

Actividad	Tiempo
Picking, inspección, empaque, despacho (por unidad)	8 s · 15 s · 20 s · 5 s
Descarga de un camión (por unidad)	1 s
Maniobra de un camión al tomar el área de carga	10 s
Reintento de un camión expulsado por deadlock	tras 2 min
Procesamiento de Inventario por mensaje	20 s + 1 s por unidad
Revisión del detector de deadlock	500 ms (tiempo real)
Envío de snapshots a la interfaz	300 ms (tiempo real)

Con la velocidad por defecto (×60), un pedido estándar de 10 unidades tarda unos 5,5 s reales. Medido con estos tiempos y la mezcla por defecto, el sistema despacha como máximo unos **2 pedidos por minuto simulado**, limitado por los 10 encargados, que pasan casi todo el tiempo ocupados.

4. Reloj simulado

Los tiempos objetivo (1, 10, 30, 60 y 120 minutos) hacen impracticable observar el sistema en tiempo real. `SimClock` convierte el tiempo real en simulado con un **factor de aceleración** (por defecto ×60: 1 s real = 1 min simulado; ajustable de ×0,1 a ×100.000) y permite **pausar**. La interfaz ofrece ×1, ×10, ×60 y ×300.

- **Todo el sistema consulta ese reloj.** Ningún componente usa `System.currentTimeMillis()` para decidir algo de la simulación. Las esperas se hacen con `SimClock.sleep`, que respeta la velocidad y la pausa y es **interrumpible** (necesario para cancelar y apagar sin esperas).
- **La pausa congela el progreso.** El avance de un pedido se calcula a partir del reloj simulado, así que se detiene sin código especial; la ocupación de los recursos también se mide con él, de modo que una pausa no cuenta como tiempo ocioso.
- **Los detectores corren en tiempo real.** El detector de deadlock y el difusor de snapshots usan tiempo real porque un deadlock no depende de la velocidad de la simulación: a ×0,1 sigue siendo un deadlock.
- **Cambiar la velocidad no distorsiona lo ya medido:** las duraciones de las actividades se expresan en tiempo simulado, así que funcionan igual a cualquier velocidad.

5. Planificación

5.1 El problema de la planificación

Cuando hay más pedidos que capacidad, el sistema debe decidir **cuál se atiende primero**. Se evalúan varias políticas (prioridad estricta, dar oportunidad a los que llevan más tiempo esperando, procesar primero los pedidos con menos artículos) y compararlas antes de decidir cuál usar. Es el problema clásico de planificación de CPU, con los pedidos como procesos, el nivel de servicio como prioridad, el número de artículos como duración estimada y los 10 encargados como «procesadores».

5.2 La lista de espera y el despachador

`WaitingList` es una **lista** (no una cola de prioridad) protegida por un `ReentrantLock` con una `Condition changed`. Elegir al mejor pedido es un recorrido O(n) con la política vigente. Es una decisión consciente: con una cola de prioridad, cambiar de política en caliente obligaría a reconstruirla; con una lista, el cambio es inmediato y la lista es pequeña frente al costo de procesar un pedido.

El `Dispatcher` (un solo hilo) aplica una **decisión tardía**: espera a que haya algún pedido elegible y algún encargado libre y *solo entonces* elige al mejor con la política vigente. Si eligiera antes, un pedido Exprés que llega mientras no hay encargado quedaría detrás de uno ya elegido. Esperar capacidad no ocupa recursos: `ResourcePool.awaitAvailableUnit()` espera sin tomar la unidad. Como el despachador es el único que asigna encargados, entre «hay uno libre» y «lo tomo» nadie puede quitárselo; esos 10 encargados limitan a 10 los pedidos en proceso.

5.3 Las cinco políticas

Cada política es un `Comparator<QueuedOrder>` (patrón *Strategy*) intercambiable en caliente y **sin apropiación** (un pedido en proceso no se interrumpe). Cuando dos pedidos empatan, siempre desempata el número de llegada.

Tabla 12. Políticas de atención: clave de orden y lo que demuestran.

Política	Clave de orden (menor = primero)	Qué demuestra
FIFO	Número de llegada	Línea base: ignora urgencia y tamaño.
Prioridad estricta	Nivel de servicio; luego llegada	Rapidez para los urgentes; **inanición** del nivel 5.
Prioridad con envejecimiento	`llegada + nivel × 10 min`	Un pedido que espera «sube» de prioridad: evita la inanición.
EDF (menor vencimiento)	`llegada + objetivo del nivel`	La tabla del enunciado ya son *deadlines*.
SJF (menos artículos)	Total de artículos; luego llegada	Libera estaciones rápido; castiga a los pedidos grandes.

Envejecimiento sin depender del tiempo actual

El envejecimiento suele escribirse como `prioridad = nivel − espera / T`, pero eso depende de «ahora»: el orden cambiaría *mientras se ordena*, y los algoritmos de ordenamiento pueden fallar

con comparadores que no son estables. Como «ahora» aparece igual a ambos lados de una comparación, se cancela:

```
nivelA − (ahora − llegadaA)/T  <  nivelB − (ahora − llegadaB)/T
        ⇔   llegadaA + nivelA·T  <  llegadaB + nivelB·T
```

Así el envejecimiento se convierte en una **clave fija por pedido** (`llegada + nivel × 10 min`), con exactamente el mismo orden pero sin depender del reloj. Todas las políticas siguen esta regla.

Orden dentro de los recursos

La política no solo ordena la lista de espera: cada `ResourcePool` ordena a sus propias esperas con el mismo criterio (`OperationOrdering`), para que un Exprés tampoco espere detrás de un Económico dentro de una estación de empaque. Hay una excepción deliberada: **los camiones van antes que los pedidos** en las filas de los pools. La recepción es finita y desbloquea el flujo de stock; si fuera al revés, bajo carga los camiones podrían no llegar a atenderse nunca y los pedidos esperarían stock para siempre.

5.4 Pedidos que esperan stock

El despachador solo admite pedidos **elegibles**: aquellos para los que hay stock en todas sus líneas. A uno al que le falta mercadería se lo salta y pasa adelante el siguiente mejor según la política; mientras espera no retiene nada. Cuando Inventario procesa una llegada avisa al planificador (`stockChanged()`), que despierta al despachador para que reevalúe la lista. El planificador nunca toca las estructuras de Inventario. Con la política vigente, un pedido grande sin stock puede ser postergado por pedidos chicos que sí tienen: es el mismo fenómeno de inanición de SJF.

5.5 Generador automático de pedidos

Se separa **qué** llega de **cuándo** llega:

- `WorkloadGenerator` decide, de forma **determinista a partir de una semilla**, qué pedido llega: el nivel según la mezcla 5/15/35/25/20 %, un cliente de ese nivel, de 1 a 3 productos distintos y cantidades de `1 + exponencial` con tope de 30. No sabe de hilos ni de relojes, así que el comparador puede reproducir la misma carga exactamente.
- `OrderGenerator` (un hilo) decide el momento. Las llegadas siguen un proceso de **Poisson**: el tiempo entre pedidos es exponencial. Para que un cambio de intensidad se note de inmediato (a intensidad baja, un intervalo ya sorteado puede ser de horas simuladas) la espera se hace en tajadas de 10 s simulados, cada una consume del trabajo sorteado a la intensidad vigente (*time-rescaling* de un proceso de Poisson no homogéneo).
- La intensidad va de 0,05 a 20 pedidos por minuto simulado. «Alta» (1,5) queda cerca del límite de capacidad (~2/min) y «Saturación» (3) lo supera: la lista de espera crece sin parar, que es lo que exige el enunciado («sin límite alguno en su cantidad»).
- Las semillas por defecto son pequeñas porque JavaScript solo representa sin pérdida enteros hasta 2^{53}.

5.6 Estadísticas

- **Espera** = desde que el pedido llega hasta que se le asigna un encargado. **Proceso** = desde que se le asigna el encargado hasta que termina (incluye esperas por recursos). Un pedido está **a tiempo** si su espera es ≤ el objetivo de su nivel.

- Por nivel: despachados, espera promedio y máxima (la máxima revela inanición), proceso promedio y porcentaje a tiempo; además totales y rendimiento (pedidos por minuto simulado).
- Cada pedido despachado produce un `CompletedOrder` que llega a `SimulationStatistics` mediante un *listener* del procesador, de modo que `processing` no depende de `stats`. Un `ReentrantLock` protege los contadores juntos para que un snapshot nunca vea, por ejemplo, la cantidad incrementada pero la suma sin actualizar.
- **Ocupación de recursos:** cada `ResourcePool` integra «unidades ocupadas × tiempo» en cada concesión y liberación. Es exacto (no es muestreo) y se mide con el reloj simulado. «Reiniciar estadísticas» reinicia contadores y ocupación a la vez.

5.7 Comparador de políticas

El enunciado pide «poder comparar el desempeño de cada política antes de decidir cuál usar en producción». La pestaña *Comparador* corre **la misma carga** con las cinco políticas y muestra los resultados lado a lado (espera promedio y máxima por nivel, porcentaje a tiempo, rendimiento y ocupación de encargados), con el mejor valor de cada fila resaltado, un resumen en frases y un botón «Usar esta política» que cambia la del sistema en vivo.

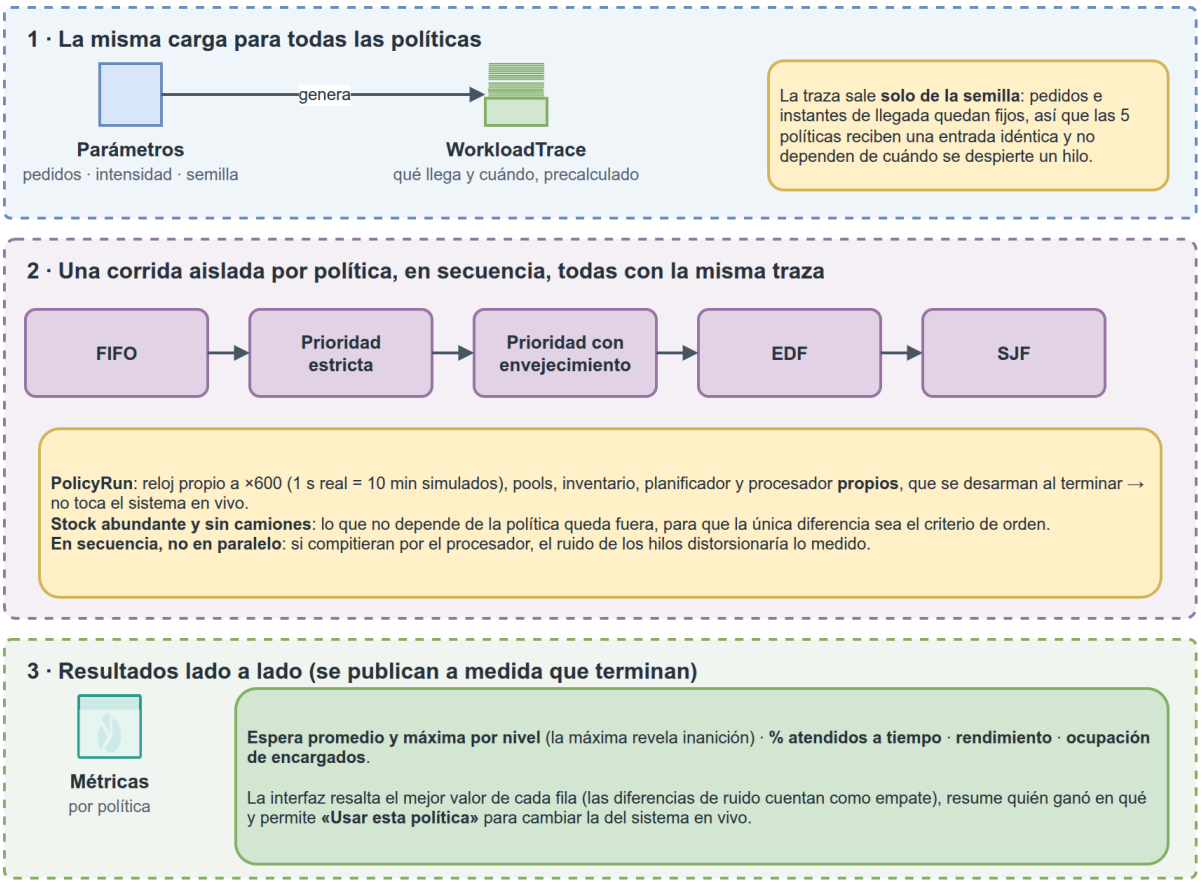


Figura 3. Funcionamiento del comparador de políticas.

Decisiones de diseño

- **Simulación aislada (`PolicyRun`).** Cada política corre con su propio reloj, pools, inventario, planificador y procesador, que se desarman al terminar; no toca los pedidos, el stock ni las estadísticas en vivo. Se excluyen a propósito lo que no depende de la política: el stock es abundante (nadie espera mercadería) y no hay camiones (no hay deadlock).
- **Misma entrada: traza precalculada (`WorkloadTrace`).** Los pedidos y sus instantes de llegada se calculan de antemano a partir de la semilla, así que las cinco políticas reciben una entrada idéntica y no dependen de cuándo se despierte un hilo.
- **En secuencia, no en paralelo.** Cada corrida usa muchos hilos; si compitieran entre sí por el procesador, el ruido de planificación de hilos distorsionaría lo medido. Se corre a ×600.
- **Segundo plano con progreso.** `BenchmarkRunner` corre en su propio hilo y publica cada resultado al terminar en un registro inmutable `volatile`; admite cancelar (interrupción, que desarma los hilos de la corrida) y rechaza una segunda comparación simultánea (409).

Reproducibilidad y su límite

No es un motor de eventos discretos sino una simulación con hilos reales, así que la planificación de hilos del sistema operativo mueve unos instantes simulados. Medido: una misma política repetida varía unos segundos sobre esperas de miles de segundos (**menos del 1 %**), muy por debajo de las diferencias entre políticas. Un test lo verifica (la misma política dos veces da resultados casi iguales) y la interfaz trata como empate las diferencias de ruido (≤ 5 s o ≤ 1 punto porcentual) en vez de coronar un «mejor».

Resultados observados

La Tabla 13 resume una corrida de referencia con carga saturada (150 pedidos, 6 pedidos por minuto, semilla 42). Los valores son esperas promedio; coinciden con lo que predice la teoría de planificación.

Tabla 13. Corrida de referencia: 150 pedidos, 6 pedidos/min, semilla 42 (carga saturada).

Política	Espera Exprés	Espera Económico	Espera promedio	A tiempo
FIFO	29 min 53 s	40 min 59 s	41 min 12 s	51 %
Prioridad estricta	**40 s**	1 h 41 min	42 min 57 s	61 %
Con envejecimiento	**37 s**	1 h 30 min	41 min 33 s	59 %
EDF	**40 s**	1 h 41 min	42 min 55 s	61 %
SJF	20 min 57 s	**16 min 57 s**	**17 min 5 s**	**76 %**

- **Prioridad estricta:** el nivel Exprés espera una fracción de lo que espera con FIFO (40 s frente a casi 30 min), pero el nivel Económico paga con más del doble de espera: es la **inanición** que el enunciado quiere que se observe.
- **Envejecimiento:** devuelve parte de esa espera a los niveles bajos (Económico baja de 1 h 41 min a 1 h 30 min) y casi no afecta a Exprés (37 s), aunque Prioritario espera más (de 2 min 16 s a 7 min 17 s): es el precio de no abandonar a nadie.
- **SJF:** minimiza la espera promedio general (resultado clásico de planificación), pero no respeta el nivel de servicio: Exprés espera 21 min, mucho más que con prioridad estricta.
- **EDF ≈ prioridad estricta** cuando todos los pedidos llegan en una ventana más corta que la diferencia entre objetivos de nivel (los plazos se ordenan igual que los niveles); se separan

con llegadas más espaciadas. El envejecimiento (escalón de 10 min por nivel) se parece a la estricta por la misma razón.

- Con carga moderada (1,5 pedidos/min) las políticas casi no se distinguen: no hay a quién preferir. La diferencia aparece cuando hay cola.

6. Sincronización y exclusión mutua

6.1 Principios

Toda la concurrencia del proyecto se rige por dos reglas, que son también convenciones del código: **el estado compartido es inmutable o se protege con un lock documentado**, y **todo comentario de sincronización explica el porqué**. En la práctica, cada clase con estado mutable declara en su Javadoc qué lock lo protege y qué invariante mantiene. Los mecanismos usados, de los más generales a los más específicos, son:

- **Monitores con `ReentrantLock` + `Condition`** donde hay que esperar una condición: `ResourcePool`, `WaitingList`, `ReceptionBuffer`, el generador y las estadísticas.
- **`ReentrantReadWriteLock`** donde hay muchas lecturas y pocas escrituras: `Inventory` y `Warehouse`.
- **Registros inmutables publicados en un campo `volatile`** donde un solo hilo escribe y otros leen: el estado observable de un pedido (`OrderExecution`) y el de una comparación (`BenchmarkRunner`). El lector obtiene una foto coherente sin tomar ningún lock.
- **Colecciones concurrentes** (`ConcurrentHashMap`, `CopyOnWriteArrayList`) y **contadores atómicos** donde la operación es independiente: registro de pedidos activos, lista de *listeners*, números de ticket.

6.2 ResourcePool: un monitor que hace de semáforo

Un `ResourcePool` representa las unidades idénticas de un recurso (por ejemplo, las 6 estaciones de empaque). Conceptualmente es un **semáforo de conteo**: `acquire()` bloquea si no hay unidades libres y `release()` devuelve una y despierta a alguien. Se implementó como un **monitor** y no con `java.util.concurrent.Semaphore` porque el proyecto necesita tres cosas que un semáforo no ofrece:

1. **Saber quién retiene y quién espera.** La interfaz debe mostrar «en uso, y por quién» y el detector de deadlock necesita las matrices de asignación y solicitud. Un `Semaphore` solo cuenta permisos.
2. **Conceder por prioridad.** Al liberarse una unidad debe recibirla el *mejor* candidato según la política de atención, no uno arbitrario ni el más antiguo.
3. **Poder expulsar a quien espera** (la víctima de un deadlock), lo que un `Semaphore` no permite.

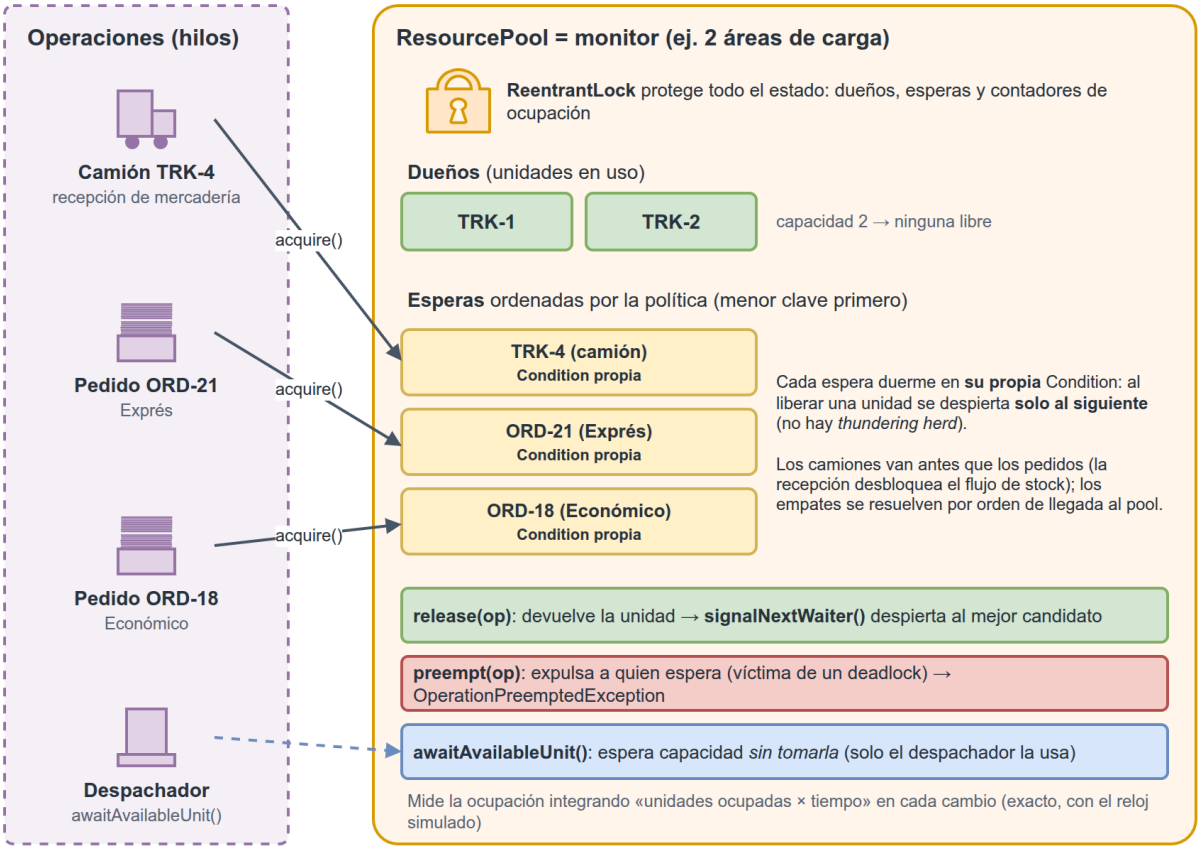


Figura 4. Estructura de un ResourcePool: dueños, esperas con su propia Condition y operaciones.

Un único `ReentrantLock` protege todo el estado del pool (dueños, esperas y contadores de ocupación). Cada operación que espera tiene **su propia `Condition`**, de modo que al liberar una unidad se despierta únicamente al siguiente en turno: no hay *thundering herd* (despertar a todos para que casi todos vuelvan a dormirse). El corazón es este fragmento de `ResourcePool`:

```
private void awaitTurn(Waiter self)
        throws InterruptedException, OperationPreemptedException {
    while (!self.preempted && !canGrant(self)) {
        self.turn.await();   // duerme en SU Condition y suelta el lock
    }
    if (self.preempted) {
        throw new OperationPreemptedException(self.operation.id(), type);
    }
}

private boolean canGrant(Waiter candidate) {
    // hay unidad libre Y este es el mejor candidato según la política
    return holders.size() < capacity
            && bestWaiter().filter(best -> best == candidate).isPresent();
}

/** Despierta al siguiente en turno solo si realmente hay una unidad libre para él. */
private void signalNextWaiter() {
    if (holders.size() < capacity) {
        bestWaiter().ifPresent(waiter -> waiter.turn.signal());
    }
}
```

Detalles que importan para la corrección:

- **El `await` va en un `while`**, no en un `if`: tras despertar hay que volver a comprobar la condición (despertares espurios y cambios de estado entre la señal y la toma del lock).
- **El bloque `finally` de `acquire`** retira al hilo de la fila y llama a `signalNextWaiter()` tanto si obtuvo la unidad como si fue interrumpido o expulsado: de lo contrario, una unidad libre podría quedar sin que nadie la reclame.
- **Los empates** entre esperas con la misma prioridad se resuelven por orden de llegada al pool (un número de ticket), lo que evita inanición entre iguales.
- **`release` valida la propiedad:** liberar algo que no se retenía lanza `IllegalStateException`, lo que convierte un error de lógica en un fallo inmediato en lugar de una corrupción silenciosa. Una operación retiene como máximo una unidad de cada pool.
- **`awaitAvailableUnit()`** espera a que haya capacidad *sin tomar la unidad* (solo la usa el despachador, sección 5.2). Se señala con una `Condition` aparte, `unitAvailable`.
- **`preempt(op)`** marca a la espera como expulsada y la despierta; su hilo recibe `OperationPreemptedException`, libera todo lo que retiene y se marcha (sección 10.7).

Finalmente, el pool mide su **ocupación** integrando «unidades ocupadas × tiempo» cada vez que esa cantidad cambia (al conceder y al liberar), en lugar de muestrear de vez en cuando. Es exacto y, al medirse con el reloj simulado, una pausa no diluye el resultado.

6.3 WaitingList

La lista de espera es el punto de encuentro entre los hilos que generan pedidos (el generador, la API, los reintentos por deadlock) y el despachador. Un `ReentrantLock` protege la lista **y la política vigente**, de modo que elegir y cambiar de política nunca se pisan. El despachador, único consumidor, duerme en la `Condition changed` dentro de `awaitEligible(predicado)` hasta que haya un pedido elegible; `add()` (un pedido nuevo o devuelto) y `reevaluate()` (llegó stock) la señalan. La elección del mejor se hace después, con `takeBest(predicado)`, lo más tarde posible.

El predicado de **elegibilidad** («¿hay stock para todas las líneas?») se evalúa **con el lock de la lista tomado**, por lo que está documentado que no debe llamar a nada que a su vez espere por la lista: solo consulta el inventario (bajo su propio lock de lectura), así que no puede formarse un ciclo de locks entre ambos.

6.4 Lectores y escritores: Inventory y Warehouse

Tanto el inventario como el almacén se consultan constantemente (cada snapshot, varias veces por segundo) y se modifican rara vez (al despachar un pedido o procesar una llegada). Es el patrón **lectores/escritores**: un `ReentrantReadWriteLock` permite que muchos lectores compartan el lock de lectura mientras que las modificaciones toman el de escritura de forma exclusiva. En `Inventory`, además, la **reserva de stock es atómica y de todo o nada**: un pedido reserva todas sus líneas en una sola operación o ninguna. Verificar y descontar ocurren bajo el mismo lock de escritura, sin hueco entre ambos, de modo que no hay sobreventa y ningún pedido retiene stock parcial esperando el resto (se elimina la condición de *retención y espera* sobre el inventario).

6.5 Resumen de mecanismos

Tabla 14. Estado compartido y mecanismo que lo protege.

Clase	Estado protegido	Mecanismo
`ResourcePool`	Dueños, esperas, ocupación	`ReentrantLock` + `Condition` por espera
`WaitingList`	Pedidos en espera y política vigente	`ReentrantLock` + `Condition changed`
`ReceptionBuffer`	Cola acotada de mensajes	`ReentrantLock` + `notFull` + `notEmpty`
`Inventory`	Existencias y reservas	`ReentrantReadWriteLock`
`Warehouse`	Ubicaciones y lotes	`ReentrantReadWriteLock`
`SimulationStatistics`	Contadores por nivel	`ReentrantLock` (contadores juntos)
`OrderExecution`	Estado observable del pedido	Registro inmutable en campo `volatile`
`BenchmarkRunner`	Estado de la comparación	Registro inmutable en campo `volatile`
`CustomerRegistry`	Cuentas de cliente	`ConcurrentHashMap` + `AtomicInteger`
`OrderGenerator`	Configuración y semilla	`ReentrantLock` + `Condition`

6.6 Cómo se prueba la concurrencia

Una prueba de concurrencia que depende de `Thread.sleep` es una prueba intermitente. Las del proyecto siguen cuatro prácticas (resultados en la sección 13): sincronización explícita con **latches** (`CountDownLatch`) en lugar de esperas arbitrarias; esperas activas con tiempo máximo (`Await.until`); **repetición** de cada prueba de concurrencia muchas veces para destapar intermitencias; y **mutaciones**: se rompe a propósito el código (por ejemplo, hacer no atómica la reserva de stock) y se comprueba que algún test falla. Una prueba que no detecta su mutación no protege nada.

7. Flujo de un pedido

Figura 5 recorre la vida de un pedido, desde que el cliente lo registra hasta que se despacha. Tiene dos tramos: la **admisión** (lista de espera y decisión del despachador) y el **procesamiento por etapas**, que ejecuta un hilo propio por pedido.

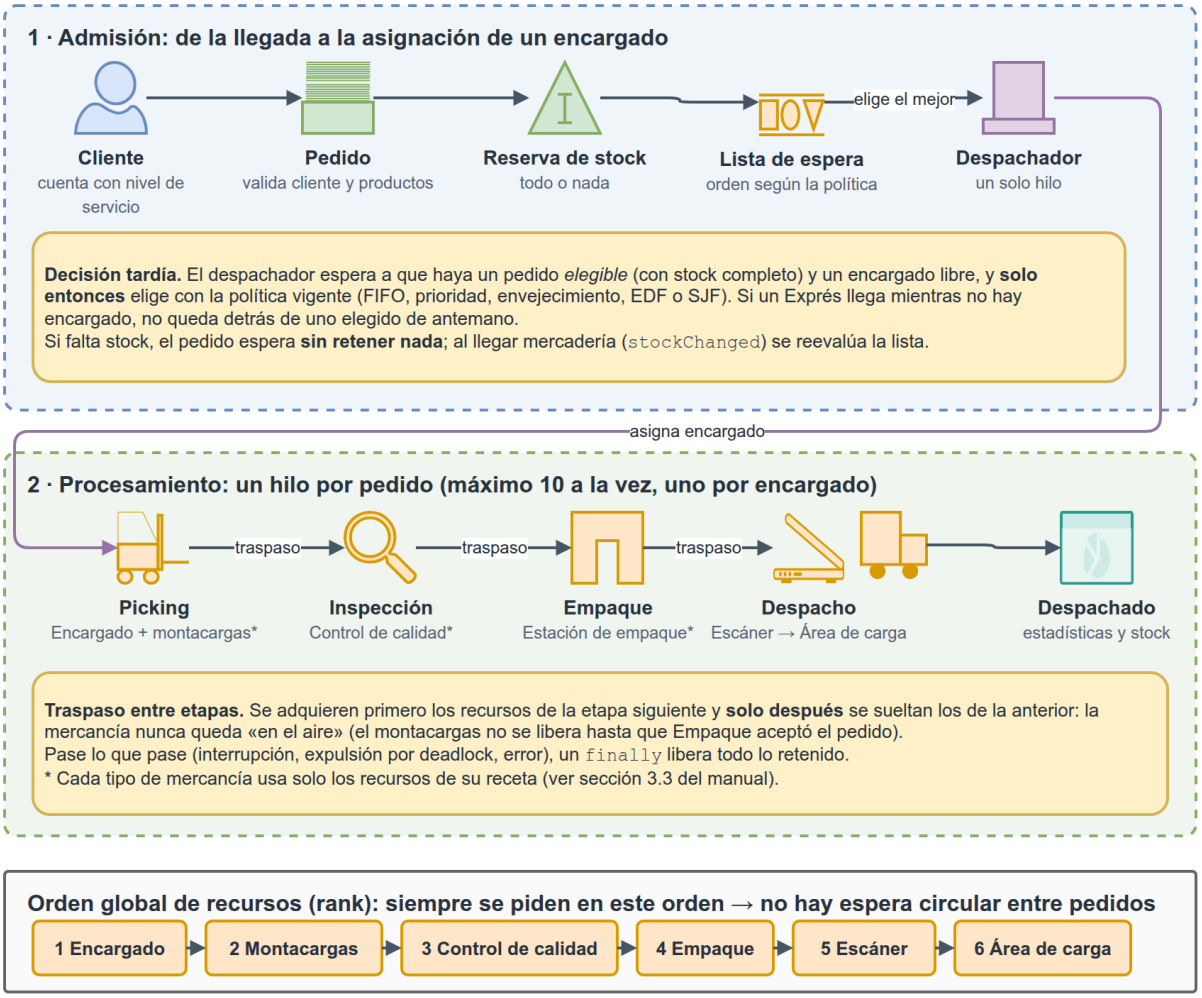


Figura 5. Ciclo de vida de un pedido: admisión y procesamiento por etapas.

7.1 Admisión

`OrderService.place` valida el pedido (cliente existente, productos del catálogo, cantidades válidas) y lo entrega al planificador, que lo deja en la lista de espera. El despachador solo considera pedidos elegibles (con stock para todas sus líneas), elige al mejor según la política cuando hay un encargado libre y, al admitirlo, **reserva todas sus líneas en una sola operación atómica** (`Inventory.reserve`). Mientras espera, el pedido no retiene nada (sección 5.2).

7.2 Etapas y modelo de retención

Todo pedido recorre cuatro etapas en orden: **picking → inspección → empaque → despacho**. Cada recurso pertenece a una etapa y el orden de las etapas coincide con el orden global de recursos. El modelo de retención de `OrderWorkflow` es:

1. El **encargado** se retiene durante todo el pedido (se asigna en la admisión).
2. Dentro de una etapa, los recursos se piden en el orden global y se retienen mientras se espera el siguiente (*hold-and-wait*, una de las cuatro condiciones de Coffman, aceptada porque entre pedidos no puede cerrarse un ciclo).
3. Al pasar de etapa se adquieren **primero los recursos de la siguiente y solo entonces se sueltan los de la anterior** (traspaso o *hand-over*). La mercancía no puede quedar «en el aire»: el montacargas no se libera hasta que la estación de empaque aceptó el pedido.
4. Pase lo que pase (interrupción, expulsión por deadlock, error) un bloque `finally` libera todo lo retenido.

Entre pedidos nunca hay espera circular, porque ninguno pide un recurso de rank menor al que ya retiene. El deadlock de la demostración ocurre *entre áreas* (Recepción contra Despacho), no entre pedidos (sección 10).

7.3 Estado observable

Cada pedido en proceso publica su estado (etapa, recurso que espera, recursos que retiene, progreso y tiempo restante) en un **registro inmutable dentro de un campo `volatile`** (`OrderExecution`). Solo el hilo del pedido lo reemplaza; el hilo que arma los snapshots lee una foto coherente sin tomar locks. El progreso se calcula a partir del reloj simulado, así que se congela al pausar. El tiempo restante estimado que muestra la interfaz es el trabajo restante de las etapas pendientes; no incluye esperas futuras por recursos, que no se pueden predecir.

8. Comunicación entre áreas: Recepción e Inventario

El enunciado pide dos áreas independientes que **no modifican las estructuras internas de la otra**: Recepción informa qué llegó y en qué cantidad, e Inventario lo procesa a su propio ritmo, incluso cuando Recepción genera información más rápido de lo que Inventario puede procesar, sin perder datos ni bloquearse indefinidamente. Es el problema clásico del **productor/consumidor con buffer acotado**.

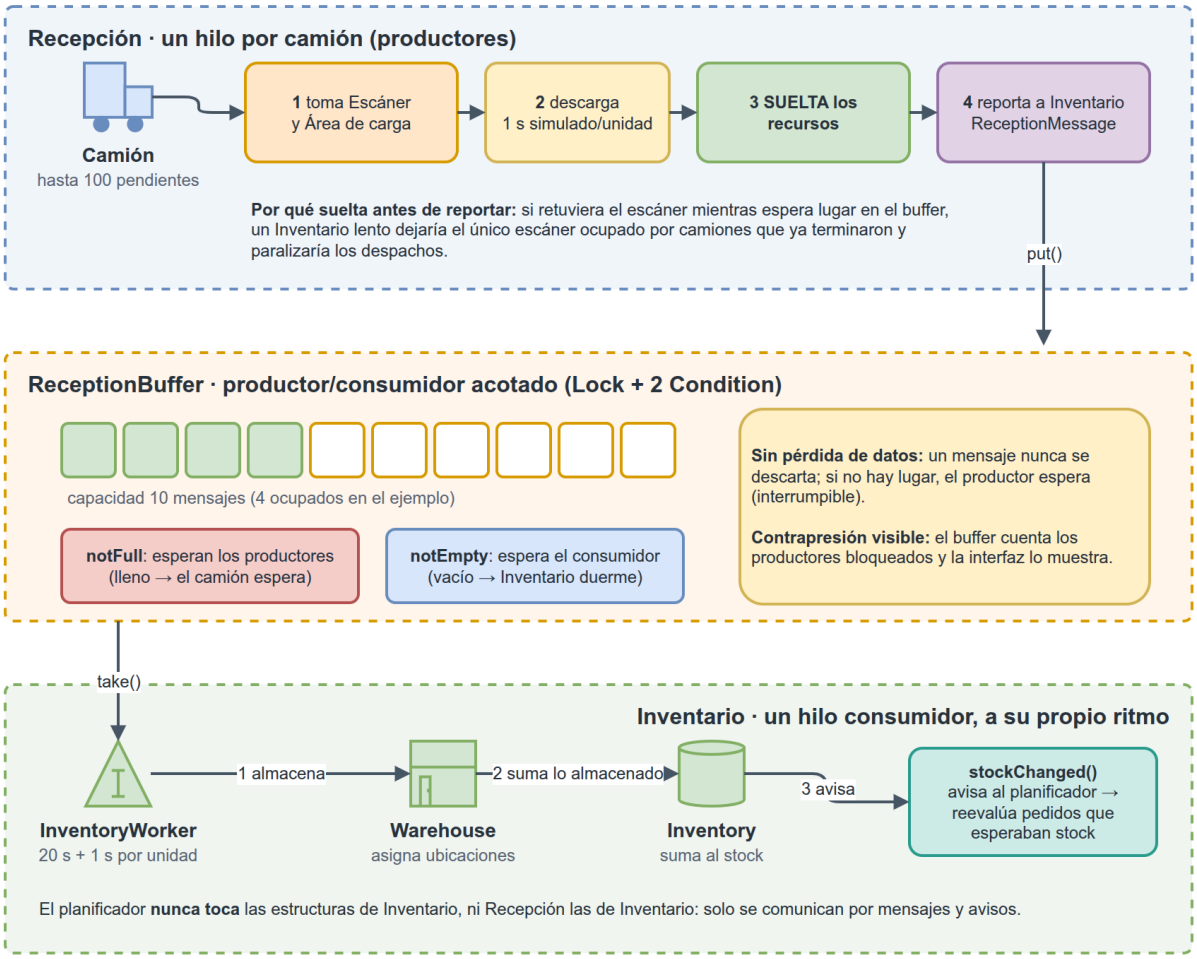


Figura 6. Recepción (productores), buffer acotado e Inventario (consumidor).

8.1 Recepción (productores)

Cada camión que llega es una operación con su propio hilo: toma el escáner y un área de carga, descarga y registra la mercadería (1 s simulado por unidad), **suelta los recursos** y recién entonces informa a Inventario con un `ReceptionMessage`. La interfaz permite registrar varios camiones iguales a la vez para provocar una ráfaga. Hay límites para que una ráfaga no cree hilos sin control: hasta 1.000 unidades por camión, 50 camiones por solicitud y 100 pendientes en total.

> **Decisión: soltar los recursos antes de reportar**
> Si el camión retuviera el escáner y el área de carga mientras espera lugar en el buffer, un Inventario lento dejaría el **único escáner** ocupado por camiones que ya terminaron de descargar, y paralizaría los despachos de pedidos. Soltándolos antes, un buffer lleno solo hace esperar al camión que reporta. Esta propiedad tiene un test que la comprueba y una mutación que la rompe (si el camión reporta antes de soltar, el test falla).

Otras dos decisiones de Recepción: pide sus recursos en el **orden global** (escáner → área de carga), igual que los pedidos, y tiene **prioridad sobre los pedidos** en las filas de los pools (sección 5.3). El orden inverso —área de carga → escáner— es el que se usa a propósito para provocar el deadlock de la demostración (sección 10).

8.2 El buffer acotado

`ReceptionBuffer` implementa el productor/consumidor **a mano** con un `ReentrantLock` y dos `Condition` (capacidad 10):

- **Sin pérdida de datos:** si está lleno, el productor *espera*; el mensaje nunca se descarta. La espera es interrumpible.
- **Sin bloqueo indefinido:** el consumidor siempre progresa y no depende de nada que el productor retenga (los camiones ya soltaron sus recursos), así que el espacio siempre termina liberándose.
- **Contrapresión visible:** el buffer cuenta los productores bloqueados y la interfaz lo muestra (el medidor del buffer y el número de camiones esperando).
- **Dos condiciones, no una:** `notFull` la esperan los productores y `notEmpty` el consumidor. Con una sola, cada `signal` podría despertar a alguien del otro grupo y perderse. Al interrumpirse un hilo que ya había sido señalado, pasa la señal al siguiente para no perder el despertar.

8.3 Inventario (consumidor)

`InventoryWorker` es un hilo propio que retira mensajes a su ritmo (20 s simulados fijos más 1 s por unidad, deliberadamente más lento que la llegada de camiones para que se vea llenarse el buffer), almacena la mercadería en el almacén, suma al stock **solo lo almacenado** y **avisa** que cambió el stock de ese producto (`stockChanged()`), lo que hace que el planificador reevalúe los pedidos que esperaban. Las existencias iniciales son 300 unidades de cada producto.

Ciclo del stock: `disponible → reservado` al admitir un pedido; al despacharlo, las unidades *salen*; si el pedido se aborta (por ejemplo, expulsado por un deadlock), *vuelven* a disponible (`StockHandling`, un *listener* del ciclo de vida del pedido). Un pedido expulsado y reintentado no descuenta dos veces: hay tests de API que lo comprueban de punta a punta.

9. Administración de almacenamiento

El enunciado pide llevar un control preciso de en qué ubicaciones físicas está cada mercadería, sabiendo que **un lote puede ocupar varias ubicaciones que no son contiguas**. Es el mismo problema que la asignación **no contigua** de memoria: la paginación. Figura 7 muestra la correspondencia.

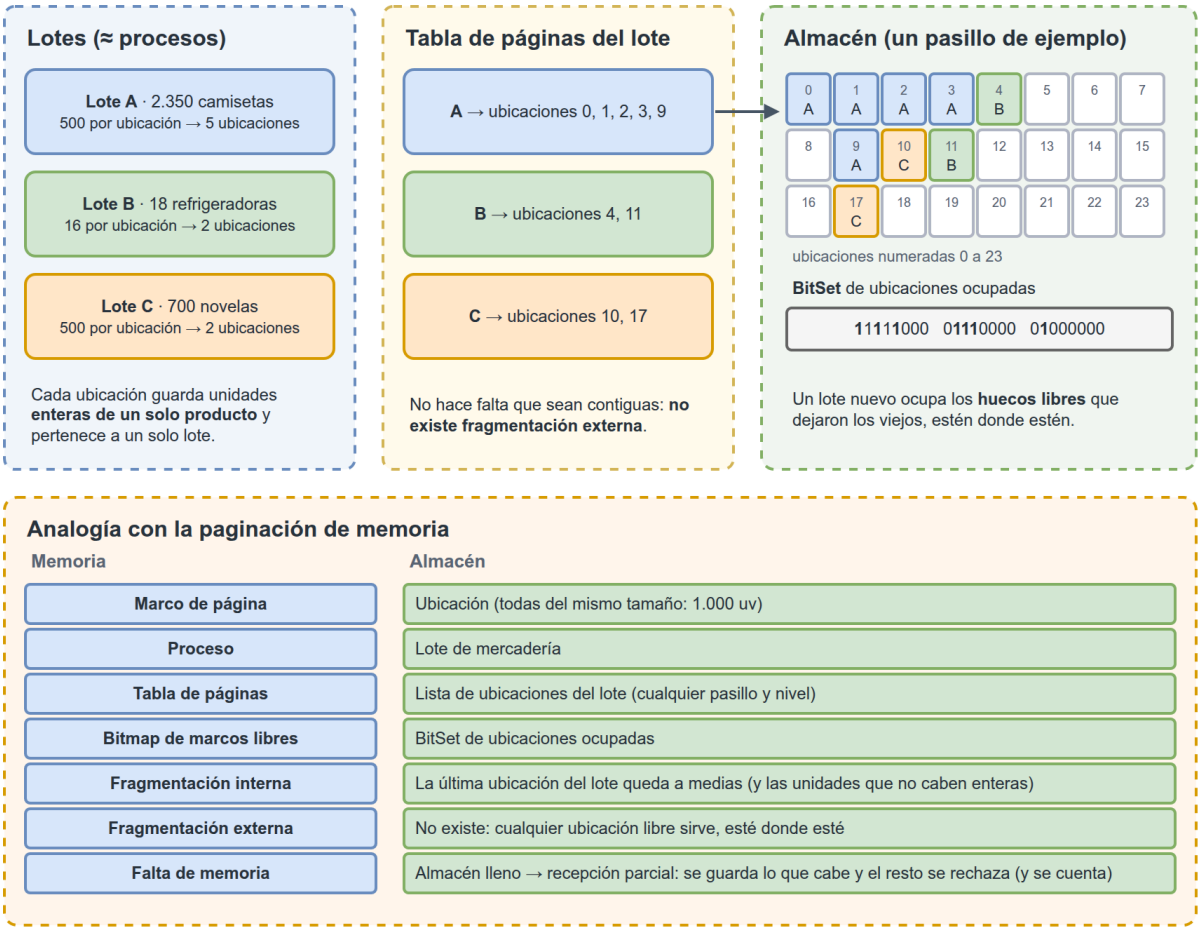


Figura 7. Asignación no contigua del almacén y su analogía con la paginación de memoria.

9.1 Dimensiones y reglas

- **6 pasillos × 8 posiciones × 5 niveles = 240 ubicaciones de 1.000 unidades de volumen.** Se numeran por pasillo, luego por posición y, lo que más rápido cambia, por nivel; dos ubicaciones son *contiguas* si sus índices son consecutivos. Se eligió ese tamaño porque 300 unidades de cada producto ocupan 112 ubicaciones (47 %), lo que deja espacio para recibir mercadería y para que se fragmente.
- **Una ubicación guarda unidades enteras de un solo producto y pertenece a un solo lote.** Caben `capacidad / volumen de la unidad` unidades, redondeado hacia abajo: un colchón (80) en una ubicación de 1.000 deja 40 de sobra, otra fuente de fragmentación interna. Una camiseta (2) cabe 500 por ubicación.

- **Fragmentación externa: no existe.** Cualquier ubicación libre sirve, esté donde esté. La **interna** sí: la última ubicación de un lote queda a medias.

9.2 Estrategias de colocación y salida

- `PlacementStrategy`, intercambiable en caliente (solo afecta a los lotes nuevos): **Primera libre** (menor índice: llena pasillo por pasillo) y **Pasillo menos ocupado** (reparte entre pasillos).
- **Orden de salida FIFO por lote:** al despachar se consumen primero los lotes más antiguos. Las ubicaciones que se vacían se liberan y los lotes nuevos las reutilizan: con el uso, un lote nuevo queda repartido en los huecos que dejaron los viejos (verificado en pruebas: 800 jarrones quedaron en las ubicaciones 0, 2-4, 7-9 y 112, cuatro tramos).

9.3 Almacén lleno

Cuando un camión trae más de lo que cabe, se **almacena lo que cabe y el resto se rechaza**; queda contado y visible («unidades rechazadas»). Se descartó esperar espacio: el espacio solo se libera cuando los pedidos consumen stock, y puede que ese stock no exista (por ejemplo, el almacén lleno de un producto y llega otro). Esperar sería un bloqueo indefinido.

9.4 Consistencia con el inventario

Dos estructuras describen lo mismo —el inventario (lo contable) y el almacén (lo físico)—, por lo que el **orden de las operaciones** es lo que las mantiene coherentes: al recibir, se almacena primero y se suma al inventario solo lo almacenado; al despachar, se descuenta del inventario primero y luego del almacén. Así las existencias contables nunca superan lo que hay físicamente. La invariante `disponible + reservado = unidades físicas` de cada producto se verifica en las pruebas de API tras entregas y despachos concurrentes.

9.5 Visualización

La pestaña *Almacén* muestra un mapa por pasillo (una columna por posición y una fila por nivel, el más alto arriba; cada ubicación se llena de abajo hacia arriba según su volumen y se colorea por tipo de mercancía), una tabla de lotes con su dispersión y, al apuntar un lote, resalta sus ubicaciones en el mapa. Las métricas son ocupación, espacio perdido por fragmentación interna, lotes dispersos y unidades rechazadas.

10. Deadlock

10.1 El requisito

El enunciado pide que el sistema pueda **llegar deliberadamente** a una situación en la que dos operaciones necesitan los mismos dos recursos compartidos *en orden inverso*, que presente en la interfaz **qué recursos están involucrados, qué operaciones se bloquean entre sí y por qué**, y que permita **resolverla a mano**: el usuario elige cuál de las dos operaciones continúa con los recursos disputados, y la otra regresa a la lista de espera. Esta sección explica cómo se cumplen esos tres requisitos y cómo, además, se previene.

10.2 El escenario

Los dos recursos en juego son el **escáner** (1 unidad) y las **áreas de carga** (2 unidades), compartidos entre Recepción y Despacho. Un **pedido en despacho** los pide en el orden global (escáner → área de carga), pero un **camión** los pide al revés (área de carga → escáner): primero llega y se acomoda en el andén, y después registra la mercadería. Con dos camiones (que ocupan las 2 áreas de carga) y un pedido que ya tiene el escáner, queda una espera circular: los tres están bloqueados, y el pedido y un camión son «las dos operaciones en conflicto». Figura 8 muestra el grafo de asignación en ese instante.

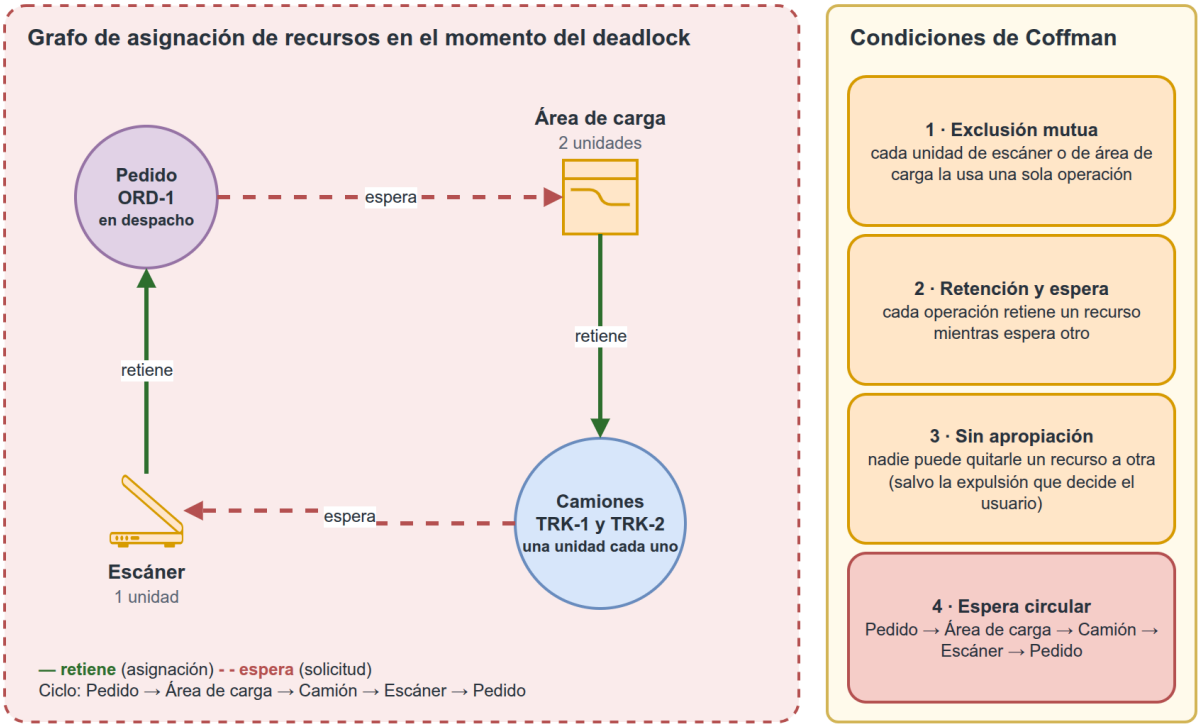


Figura 8. Grafo de asignación de recursos en el deadlock y las cuatro condiciones de Coffman.

Para que ocurra hace falta que el pedido tome el escáner *mientras* los camiones ya tienen las áreas de carga pero aún no lo han pedido. Por eso el camión hace una **maniobra** de 10 s simulados justo después de tomar el área de carga (acomodarse), y el **escenario de demostración** (`DeadlockScenario`) registra el pedido y, `T - maniobra/2` después (T = tiempo que el pedido tarda en llegar a despacho), los dos camiones. Todo es en tiempo simulado, así que funciona igual

a cualquier velocidad. Exige el sistema en reposo (sin generador, pedidos ni camiones en curso) y devuelve 409 si no lo está.

¿Puede ocurrir sin el escenario?

Sí, pero depende de cómo lleguen los camiones. Con la prevención desactivada, el generador saturado y camiones llegando **de a uno a intervalos aleatorios**, el deadlock apareció solo (a los 7 s reales, con 6 operaciones bloqueadas). En cambio **no** aparece con una ráfaga de camiones ya encolados: los camiones pasan antes que los pedidos en la fila del escáner y salen uno tras otro, así que el escáner nunca queda libre para que un pedido lo tome mientras los camiones ya tienen las áreas de carga. El deadlock necesita ese hueco. Para provocarlo a mano de forma confiable: velocidad ×1, un pedido de 1 unidad de un producto perecedero y, unos 15 s después, una llegada de 2 camiones (se detecta a los ~26 s); la interfaz lo explica en el panel.

10.3 Las cuatro condiciones de Coffman

Un deadlock exige las **cuatro a la vez**, y la interfaz las muestra en cada caso con las operaciones reales. Cada una se manifiesta así en REDXela:

Tabla 15. Condiciones de Coffman en REDXela y qué ataca cada defensa.

Condición	Cómo se manifiesta	Defensa
Exclusión mutua	Cada unidad de escáner o de área de carga la usa una sola operación.	No se puede evitar (el recurso es físico).
Retención y espera	Cada operación retiene un recurso mientras espera otro (el camión retiene el área mientras pide el escáner).	Aceptada: es parte del flujo en etapas.
Sin apropiación	Nadie puede quitarle un recurso a otra operación.	**Se rompe en la recuperación:** `preempt` expulsa a la víctima.
Espera circular	Pedido → Área de carga → Camión → Escáner → Pedido.	**Se rompe en la prevención:** orden global de recursos.

10.4 Detección

Figura 9 resume el ciclo completo: detección, recuperación manual y prevención. La detección corre en un hilo propio (`DeadlockDetector`) que cada 500 ms lee los pools y entrega los datos a `DeadlockAnalyzer`.

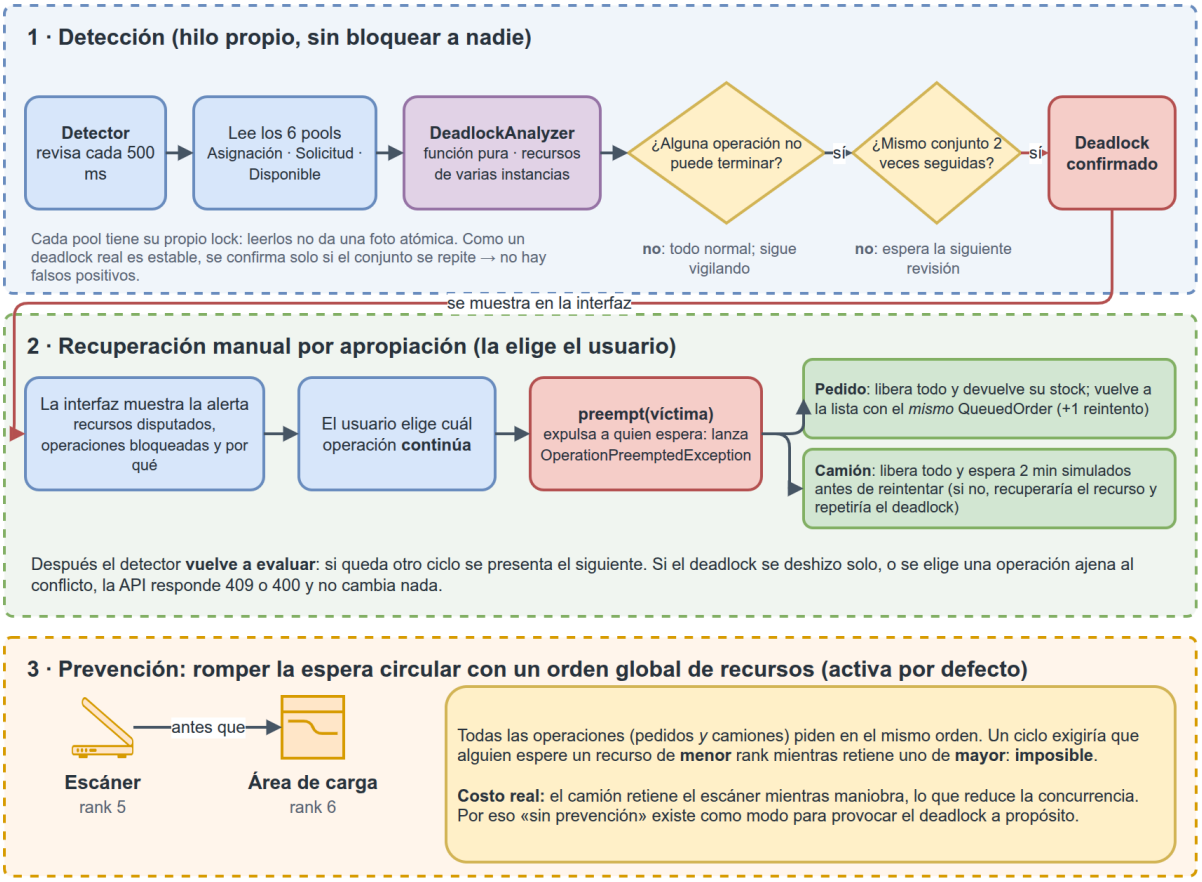


Figura 9. Detección, recuperación y prevención del deadlock.

Por qué no basta un grafo de espera

Con un recurso de **una sola unidad** por tipo, un ciclo en el grafo de espera equivale a un deadlock. Pero aquí los recursos tienen **varias unidades** (2 áreas de carga, 6 estaciones…), y entonces un ciclo es solo una condición *necesaria*: otra unidad libre podría romperlo. Por eso se usa el algoritmo de detección para recursos de varias instancias (Silberschatz, sección 7.6.2), con tres estructuras: **Asignación** (qué retiene cada operación), **Solicitud** (qué espera) y **Disponible** (unidades libres de cada recurso). Hay un test que demuestra el caso: un ciclo con una unidad libre no es deadlock.

```
trabajo ← Disponible
terminadas ← ∅
repetir
    para cada operación p ∉ terminadas:
        si p no retiene nada  o  Solicitud[p] ≤ trabajo:
            trabajo ← trabajo + Asignación[p]     // p terminaría y devolvería lo que
retenía
            terminadas ← terminadas ∪ {p}
hasta que no haya cambios
bloqueadas ← operaciones \ terminadas              // las que nunca podrán terminar
```

`DeadlockAnalyzer` es una **función pura**: recibe datos y devuelve el resultado, sin hilos ni estado, por lo que se prueba con total precisión y sin concurrencia. El informe distingue los recursos «involucrados» (los **disputados**, que las operaciones bloqueadas esperan; el encargado de bodega, que todo pedido retiene pero nadie espera, no forma parte del conflicto) y elige el **par en conflicto**: primero dos operaciones que se esperen mutuamente (la forma clásica); si no las hay, el primer tramo del ciclo más corto. Siempre existe un ciclo entre las operaciones bloqueadas.

10.5 Confirmación por estabilidad

Cada pool tiene su propio lock, así que leer los seis **no da una foto atómica** y podría armar un ciclo que nunca existió (un falso positivo). Un deadlock real, en cambio, es **estable**: nadie del conjunto bloqueado avanza. El detector declara un deadlock solo si el *mismo conjunto de operaciones* aparece en **dos revisiones consecutivas** (cada 500 ms). Esta decisión evita tomar un único lock global sobre todo el sistema, que habría frenado a todos los hilos en cada revisión. Una prueba de estrés con 8 hilos compitiendo bajo el orden global y el detector escaneando sin parar verifica que no confirma nada.

10.6 Prevención

De las cuatro condiciones se ataca la que se puede evitar: la **espera circular**. Con un **orden global de recursos**, todas las operaciones piden en el mismo orden (escáner antes que área de carga), y un ciclo es imposible: exigiría que alguien espere un recurso de *menor* rank mientras retiene uno de *mayor*. Implementarlo es ordenar las solicitudes del camión por rank (`AcquisitionPolicy` con `PreventionMode.GLOBAL_ORDER`).

- **Costo real:** el camión retiene el escáner mientras maniobra, lo que reduce la concurrencia. Es el precio de la garantía.
- **Activa por defecto**, para que el sistema no pueda congelarse por accidente; «Sin prevención» es el modo que permite el deadlock deliberado.
- «Repetir con orden global» corre el mismo escenario con la prevención activa y demuestra que no ocurre; el panel informa el estado del escenario (en curso, terminado sin deadlock, terminado con deadlock resuelto) y muestra el escáner y las áreas de carga en vivo.

10.7 Recuperación manual por apropiación

El usuario elige cuál de las dos operaciones continúa; la otra es **expulsada** (`ResourcePool.preempt`): su hilo recibe `OperationPreemptedException`, libera todo lo que retiene y «vuelve a la lista de espera». Lo que ocurre con la víctima depende de qué sea:

- **Pedido:** libera todo (incluido el encargado), devuelve su stock reservado y vuelve a la lista con el **mismo `QueuedOrder`**: conserva su hora y número de llegada (no se lo «castiga» por un deadlock que no causó) y suma un reintento. El despachador lo vuelve a admitir según la política y vuelve a reservar: no hay doble descuento de stock.
- **Camión:** libera todo y espera 2 min simulados antes de reintentar. Si reintentara de inmediato, como los camiones pasan antes que los pedidos en las filas, recuperaría el recurso que acaba de soltar y volvería a formar el deadlock.

Si el deadlock se deshizo solo antes de decidir, o el usuario elige una operación ajena al conflicto, la API responde 409 o 400 y no cambia nada. Después de resolver, el detector vuelve a evaluar: si queda otro ciclo, se presenta el siguiente.

11. Interfaz web

11.1 Qué muestra

La interfaz cumple la lista mínima del enunciado y añade vistas propias. La cabecera tiene el reloj (velocidad y pausa), el estado de la conexión y el botón «Reiniciar simulación» (con confirmación). La columna lateral reúne las entradas manuales: generador automático, registro de pedidos, llegada de mercadería y creación de clientes. El contenido principal se organiza en seis pestañas:

Tabla 16. Pestañas de la interfaz y requisito que cubren.

Pestaña	Contenido	Requisito del enunciado
Resumen	Estadísticas generales y por nivel; tarjeta de cada recurso con unidades en uso, dueños y esperas	Estado de cada recurso; estadísticas
Pedidos	Lista de espera con el criterio de orden y selector de política; pedidos en proceso con etapa, progreso y tiempo restante	Pedidos en proceso; lista de espera
Inventario	Camiones en Recepción, medidor del buffer, estado del consumidor y tabla de existencias	Registro de mercadería; coordinación
Almacén	Mapa por pasillo, lotes y dispersión, estrategia de colocación	Almacenamiento
Deadlock	Alerta, escenario de demostración, prevención, explicación (4 condiciones) y resolución	Visualizar y resolver el conflicto
Comparador	Comparación de las cinco políticas con la misma carga	Comparar políticas

Cuando se confirma un deadlock aparece un aviso visible desde cualquier pestaña, con un botón para ir directamente al panel de resolución.

11.2 Atomic Design

Los componentes se organizan en cinco niveles, y la regla es que **cada nivel solo importa de niveles inferiores** (un átomo no importa moléculas). Los componentes de presentación **no hacen fetch ni abren sockets**: eso es trabajo de `hooks/` y `services/`. La lógica reutilizable vive en funciones puras de `utils/` con tests; los estilos usan CSS Modules y solo los tokens de `styles/tokens.css` (que incluye el modo oscuro).

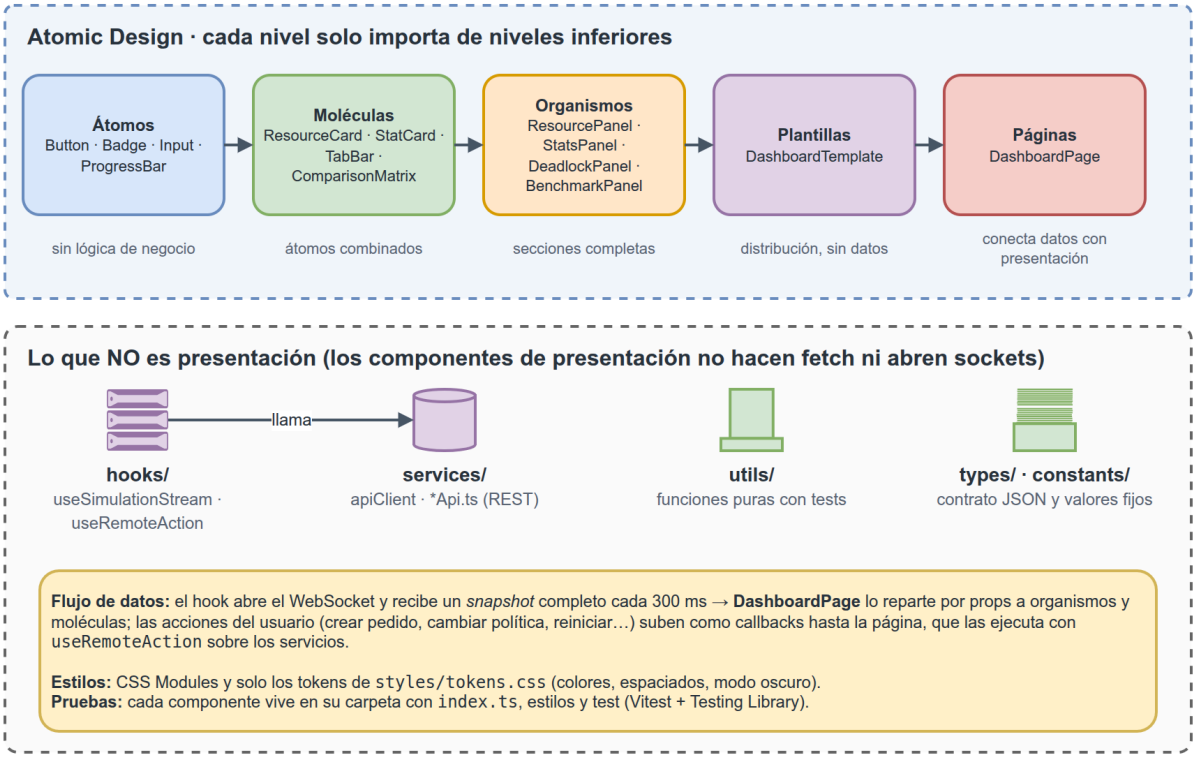


Figura 10. Organización del frontend: niveles de Atomic Design y capas de apoyo.

11.3 Flujo de datos

`useSimulationStream` abre el WebSocket, guarda el último snapshot y, si la conexión se cae, **reintenta cada 2 s** hasta que el componente se desmonte (así sobrevive a un reinicio del servidor). `DashboardPage` reparte el snapshot por props. Las acciones del usuario suben como callbacks hasta la página, que las ejecuta con `useRemoteAction`: un envoltorio que gestiona el estado de carga, el error y el resultado de una llamada al backend. Las funciones puras de `utils/` (formato de duraciones, explicación de un conflicto, resumen de la comparación, construcción del mapa del almacén) concentran la lógica que se prueba sin renderizar nada.

11.4 Accesibilidad y presentación

- Roles y etiquetas ARIA (`role="alert"` para avisos y errores, `role="progressbar"`, grupos con nombre), de modo que las pruebas consultan la interfaz como lo haría un lector de pantalla.
- El resaltado del mejor valor en las tablas comparativas no depende solo del color: lleva un texto oculto «(mejor)».
- Diseño adaptable: la columna lateral pasa debajo del contenido en pantallas angostas; colores y espaciados salen de tokens, con tema claro y oscuro según la preferencia del sistema.

12. API REST y WebSocket

Todos los grupos de rutas implementan `ApiRoutes` y se registran en `ApiServer`, que traduce las excepciones del núcleo a códigos HTTP: `IllegalArgumentException` → **400**, `StateConflictException` → **409**.

Tabla 17. Endpoints.

Método y ruta	Descripción	Respuestas
`GET /api/health`	Comprobación de vida	200
`GET /api/catalog`	Productos, tipos, niveles, políticas, estrategias y modos de prevención	200
`GET, POST /api/customers`	Listar clientes / crear una cuenta con nivel de servicio	200, 201, 400
`POST /api/orders`	Registrar un pedido manual	201, 400
`POST /api/receptions`	Registrar llegada de mercadería (producto, cantidad, camiones)	202, 400
`GET, PATCH /api/clock`	Leer / cambiar velocidad y pausa	200, 400
`GET, PUT /api/generator`	Estado / encender, intensidad y semilla del generador	200, 400
`GET, PUT /api/scheduling/policy`	Política de atención vigente / cambiarla en caliente	200, 400
`GET, PUT /api/warehouse/strategy`	Estrategia de colocación vigente / cambiarla	200, 400
`POST /api/stats/reset`	Reiniciar estadísticas y ocupación	204
`GET /api/deadlock`	Estado de la prevención y deadlock confirmado	200
`PUT /api/deadlock/prevention`	Cambiar el modo de prevención	200, 400
`POST /api/deadlock/scenario`	Lanzar el escenario de demostración	202, 409
`POST /api/deadlock/resolve`	Resolver eligiendo la operación que continúa	200, 400, 409
`GET, POST, DELETE /api/benchmark`	Estado / lanzar / cancelar la comparación de políticas	200, 202, 204, 400, 409
`POST /api/simulation/reset`	Reiniciar toda la simulación	202
`WS /ws`	Snapshot completo del sistema cada 300 ms	—

El snapshot (`SimulationSnapshot`) contiene el reloj, la política y la lista de espera (truncada a 50), el generador, las estadísticas, el inventario con el buffer y el consumidor, la recepción, el almacén, el deadlock y el escenario, el comparador, los recursos, los pedidos activos y la cantidad de pedidos completados.

13. Estrategia de pruebas y resultados

13.1 Cobertura

El proyecto tiene **352 tests en el backend** (JUnit 5 + AssertJ) y **333 en el frontend** (Vitest + Testing Library). Se ejecutan sin intermitencias: la suite completa del backend se corrió repetidamente con el directorio de compilación limpio, y las pruebas de concurrencia se repitieron decenas de veces durante su desarrollo.

Tabla 18. Tests del backend por paquete.

Paquete	Tests	Qué cubren
`api`	65	Cada ruta contra un servidor real; deadlock y reinicio de punta a punta con WebSocket
`processing`	58	Etapas, traspaso de recursos, despachador, expulsión de pedidos
`warehouse`, `deadlock`	31 + 31	Paginación, estrategias, invariantes; analizador (función pura), detector y recuperación
`inventory`	30	Reserva atómica sin sobreventa, buffer productor/consumidor, consumidor
`scheduling`, `generator`	19 + 18	Cada política, cambio en caliente; determinismo del generador por semilla
`benchmark`	17	Traza reproducible, orquestador con ejecutor falso, corridas reales
`resources`, `sim`, `domain`	16 + 14 + 14	Monitor del pool, reloj simulado, recetas del dominio
`reception`, `orders`, `stats`, `catalog`, `customers`	12 + 11 + 7 + 5 + 4	Camiones, validación de pedidos, estadísticas, catálogo, cuentas

13.2 Técnicas para probar concurrencia

- **Sin `Thread.sleep` arbitrarios:** se coordinan los hilos con `CountDownLatch` y se espera con `Await.until(condición)` con tiempo máximo.
- **Repetición:** una prueba de concurrencia pasa o falla por azar; se corre muchas veces seguidas para destapar el azar.
- **Pruebas con el sistema real:** los tests de API arrancan `ApplicationComponents` en un puerto libre y observan el WebSocket, de modo que prueban el cableado verdadero.
- **Reloj controlado:** pausar o acelerar el `SimClock` hace los escenarios deterministas (por ejemplo, congelar el reloj para que nada nuevo se complete tras un reinicio de estadísticas).
- **Pruebas de estrés dirigidas:** 16 hilos reservando stock a la vez con una barrera de salida y 8 000 intentos (sin sobreventa); 8 hilos compitiendo bajo el orden global con el detector escaneando (sin falsos positivos).

13.3 Pruebas de mutación manuales

Para saber si una prueba realmente protege algo, se **rompió a propósito el código** y se comprobó que algún test fallaba. Las mutaciones aplicadas fueron:

Tabla 19. Mutaciones aplicadas y test que las detecta.

Mutación	Propiedad que protege
Hacer no atómica la reserva de stock	Sin sobreventa bajo concurrencia (la primera versión de la prueba no la detectaba; se reforzó)
El camión reporta antes de soltar los recursos	Un buffer lleno no paraliza los despachos
`PolicyRun` ignora la política	El comparador distingue políticas
El reinicio no cierra la simulación anterior	Los reinicios no dejan hilos vivos

13.4 Prueba de aguante

Para comprobar que el sistema no se degrada, se dejó correr durante 12 minutos con 256 MB de heap, el reloj a ×600 (≈2 horas simuladas) y el generador en saturación, con llegadas de camiones, cambios de política y reinicios de estadísticas durante la corrida, midiendo cada ≈15 s con *garbage collection* forzado.

Tabla 20. Resultados de la prueba de aguante (12 min, ×600, 256 MB de heap).

Medida	Inicio	Final
Pedidos en espera	182	17.284
Pedidos despachados	199	3.878
Heap usado tras GC	8 MB	9,5 MB (rango 8–32 MB)
Hilos de la JVM	52	49 (rango 46–53)
Tamaño del snapshot	25 KB	18 KB (acotado)
Latencia de `/api/health`	1,6 ms	1,5 ms
Errores o advertencias en el log	—	0

Conclusión: con 17.284 pedidos acumulados el heap se mantuvo entre 8 y 32 MB, sin tendencia a crecer; los hilos y el tamaño del snapshot se mantuvieron estables. La lista de espera crece, como debe, pero cada pedido en espera es un objeto pequeño y no retiene recursos.

13.5 Defectos que encontraron las pruebas

Tabla 21. Defectos reales detectados durante el desarrollo y su corrección.

Defecto	Corrección
La prueba de sobreventa no detectaba una reserva no atómica	Se reforzó (16 hilos, barrera de salida, 8 000 intentos) hasta que la mutación fallara
Una carrera latente en una prueba de Inventario (comprobaba el estado antes de que el consumidor empezara)	Esperar también a que `processing` no sea nulo
El informe de deadlock listaba al encargado de bodega como «involucrado»	Los recursos involucrados son solo los disputados (los que se esperan)
`OrderProcessor.close()` devolvía con hilos de pedidos aún vivos (detectado al desarmar la simulación de cada política)	`close()` espera la terminación de los hilos (con tiempo máximo)

Defecto	Corrección
La respuesta del reinicio se cortaba si el servidor se apagaba de inmediato	Margen documentado de 300 ms antes de apagarlo
Javalin no arrancaba sin la carpeta `static` (clon sin frontend construido)	Los archivos estáticos son opcionales; solo se sirven si existen

14. Decisiones de diseño y alternativas descartadas

Resumen de las decisiones principales. Cada una se justifica en la sección indicada.

Tabla 22. Decisiones de diseño.

Decisión	Alternativa descartada	Motivo	Sec.
Javalin	Spring Boot	Spring oculta los hilos; con Javalin solo existen los del proyecto	2.2
Pools como monitores propios	`java.util.concurrent.Semaphore`	Hay que saber quién retiene y espera, conceder por prioridad y expulsar	6.2
Lista con recorrido O(n)	Cola de prioridad	La política cambia en caliente; no hay que reconstruir la estructura	5.2
Decisión tardía en el despachador	Elegir el pedido antes de tener encargado	Evita la inversión de prioridad	5.2
Claves de orden independientes del tiempo	Envejecimiento `nivel − espera/T`	Un comparador estable no depende de «ahora»	5.3
Camiones antes que pedidos en los pools	Misma prioridad que los pedidos	Si no, bajo carga el stock nunca llegaría	5.3
Reserva de stock de todo o nada	Reserva parcial por línea	Elimina retención y espera sobre el inventario	6.4
Traspaso entre etapas	Soltar y luego pedir	La mercancía no queda «en el aire»	7.2
Soltar recursos antes de reportar	Reportar reteniéndolos	Un buffer lleno paralizaría el escáner	8.1
Buffer con dos Condition	Una sola Condition	Un `signal` podría despertar al grupo equivocado	8.2
Almacén lleno: rechazo parcial	Esperar espacio	Esperar sería un bloqueo indefinido	9.3
Estado «esperando stock» derivado	Estado guardado	Nunca queda desactualizado	3.5
Confirmar el deadlock en 2 revisiones	Un lock global para leer todo	No frena al sistema y evita falsos positivos	10.5
Prevención activa por defecto	Prevención desactivada	El sistema no puede congelarse por accidente	10.6
Recuperación por apropiación	Abortar a la víctima	El pedido conserva su turno; no se pierde trabajo ni stock	10.7
Snapshot completo por WebSocket	Eventos incrementales	Robusto ante reconexiones y pérdidas	2.4
Reiniciar reconstruyendo	Limpiar componente por componente	Correcto por construcción; sin carreras	2.5
Comparador aislado y en secuencia	Medir en vivo o en paralelo	Misma carga y sin ruido entre corridas	5.7

15. Limitaciones y trabajo futuro

- **Simulación con hilos reales, no por eventos discretos.** Da fidelidad (hay concurrencia de verdad) a costa de un ruido de planificación de hilos de segundos simulados. Un motor de eventos discretos haría las corridas del comparador exactamente reproducibles.
- **Sin persistencia.** Todo vive en memoria; solo las cuentas de cliente sobreviven a un reinicio de la simulación, no al de la JVM.
- **Una sola JVM, sin autenticación.** Es una herramienta de demostración local, no un servicio.
- **La prevención cubre el par escáner / área de carga.** Es el conflicto que pide el enunciado; extender el orden global a nuevos recursos compartidos es directo porque el orden ya está en `ResourceType`.
- **Políticas sin apropiación.** Un pedido en proceso no se interrumpe aunque llegue uno más urgente; una variante con apropiación exigiría guardar y reanudar el trabajo por etapa.
- **El comparador excluye stock y camiones por diseño.** Para evaluar el efecto conjunto de la política con falta de stock, habría que añadir un modo con carga de reposición.
- **Mejoras posibles:** evitación de deadlock con el algoritmo del banquero para comparar con la prevención; más estrategias de colocación en el almacén (por afinidad de productos); exportar los resultados del comparador.

Apéndice A. Compilación, ejecución y pruebas

Requisitos: Java 21, Maven 3.9+ y Node 20.19+ (solo para construir la interfaz).

Una sola aplicación

```
cd backend
mvn package                  # construye también el frontend y genera target/redxela.jar
java -jar target/redxela.jar # abre http://localhost:7070
```

`mvn package -DskipFrontend` empaqueta sin la interfaz (el JAR solo sirve la API). Otro puerto: `java -Dredxela.port=8080 -jar target/redxela.jar`.

Desarrollo con recarga en caliente

```
cd backend  && mvn exec:java -Dexec.mainClass=com.redxela.RedxelaApplication
            # API en :7070
cd frontend && npm install && npm run dev
            # UI en :5173
```

Pruebas

```
cd backend  && mvn test
cd frontend && npm test && npm run lint && npx tsc -b
```

Solución de problemas

`Unresolved compilation problem` al correr los tests del backend. La extensión de Java de VS Code (`redhat.java`) y Maven comparten la carpeta `backend/target/`; si el editor deja clases compiladas a medias, Maven las da por buenas y los tests fallan aunque el código esté bien. Se arregla borrando lo compilado: `rm -rf backend/target/classes backend/target/test-classes`.

Estructura del repositorio

```
SO2-Proyecto/
├── backend/            Java 21 + Maven + Javalin (com.redxela)
│   └── src/{main,test}/java/com/redxela/…
├── frontend/           React + TypeScript + Vite (Atomic Design)
├── docs/
│   ├── PROPUESTA.md    diseño acordado
│   └── diagramas/      fuentes .drawio y .png de los diagramas
└── reference/          enunciado del proyecto
```

Apéndice B. Glosario

Tabla 23. Términos usados en este manual.

Término	Significado
Monitor	Construcción de sincronización que agrupa un lock y una o más condiciones para esperar a que se cumpla una propiedad del estado compartido.
Semáforo de conteo	Contador de permisos con operaciones *wait* (bloquea si es 0) y *signal*. `ResourcePool` es uno con identidad y prioridad.
Thundering herd	Despertar a todos los hilos en espera cuando solo uno puede continuar.
Hold-and-wait	Retener un recurso mientras se espera otro; una de las condiciones de Coffman.
Espera circular	Cadena cerrada de operaciones donde cada una espera un recurso que retiene la siguiente.
Prevención / evitación / detección	Impedir una condición de Coffman por diseño / no conceder solicitudes que llevan a estados inseguros / permitir el deadlock y reconocerlo para recuperarse.
Apropiación (preemption)	Quitarle un recurso a una operación; aquí, expulsar a la víctima de un deadlock.
Inanición	Una operación espera indefinidamente porque otras siempre pasan antes.
Envejecimiento (aging)	Subir la prioridad de lo que lleva esperando para evitar la inanición.
EDF / SJF	Earliest Deadline First (menor vencimiento primero) / Shortest Job First (menor trabajo primero).
Productor / consumidor	Problema de comunicar dos actividades de distinto ritmo mediante un buffer acotado.
Paginación	Asignación no contigua de memoria en marcos del mismo tamaño; aquí, de ubicaciones del almacén.
Fragmentación interna / externa	Espacio perdido dentro de una unidad asignada / espacio libre inutilizable por no ser contiguo.
Snapshot	Foto completa e inmutable del estado del sistema que el servidor envía a la interfaz.
uv	Unidad de volumen: medida de espacio de un producto y de la capacidad de una ubicación.